

CS222

Lecture 09: Synchronization

2nd Semester 2025

Kasidit Chanchio

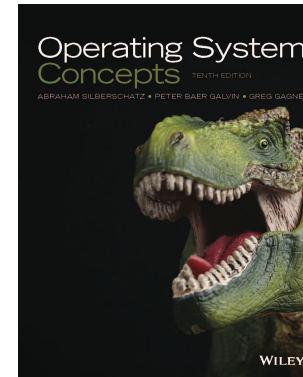
Department of Computer Science

Faculty of Science and Technology

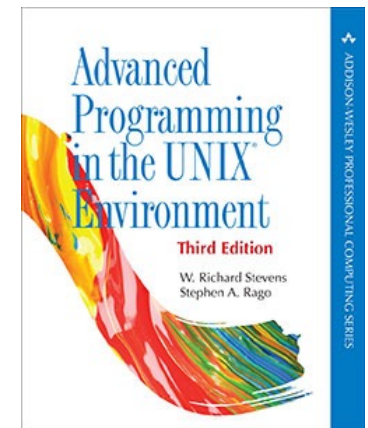
Thammasat University

Textbooks & Codes

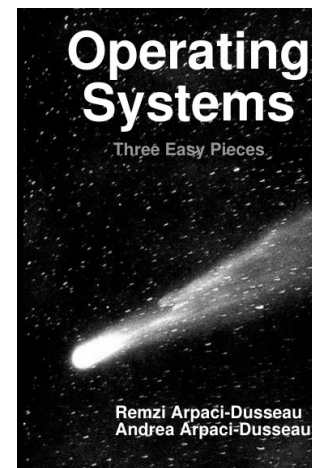
- Avi Silberschatz, Peter B. Galvin and Greg Gagne; Operating System Concepts, 10th Edition; John Wiley & Sons, Inc; 2018;
- <https://archive.org/details/silberschatz-operating-system-concepts-10e-2018>



- Advanced Programming in the UNIX Environments
- <http://www.apuebook.com/>



- Operating Systems: Three Easy Pieces
- <https://pages.cs.wisc.edu/~remzi/OSTEP/>



เครื่องมือ

- มีหลายอย่างเรา ได้กล่าวถึงหลักการคือ
 - mutex
 - Semaphore
 - Monitor
- เราจะพิจารณา
 - mutex
 - Semaphore

Mutex lock

- mutex คือการล็อก (lock) การเข้าใช้งาน shared resource และปลดล็อก (unlock) เมื่อใช้งาน resource นั้นเสร็จแล้ว
- ถ้ามีเธรดหนึ่งได้ lock เธรดอื่นที่จะเข้าใช้ resource ต้องรอ (block) จนกว่าเธรดที่ได้ lock จะ unlock
- ในกรณีที่มีเธรด blocked มากกว่า 1 เธรด เมื่อมีการ unlock เธรดที่ถูก block อยู่ทุกเธรดจะเปลี่ยนสถานะเป็น runnable แต่จะมีเพียงเธรดเดียวเท่านั้นที่ได้ lock และเธรดที่เหลือต้องกลับไป block รอเหมือนเดิม

Mutex unlock

- ในกรณีที่เธรด T1 lock A แล้ว มีเธรดอื่น T2 ... TN รอให้ unlock เมื่อ T1 unlock A เธรดหนึ่งใน T2 ... TN จะ lock ได้เท่านั้น
- ถ้าเธรด T1 ใช้คำสั่ง unlock A แต่ขณะนั้นไม่มีเธรดอื่นใด lock A การ unlock นั้นจะไม่ส่งผลอะไรกับ การ lock หรือ unlock ในอนาคต
 - ไม่มีสถานะของการ unlock
- ตาม POSIX standard เธรด T1 lock mutex variable (สมมุติว่า เป็นตัวแปร A) เธรด T1 ต้องเป็นผู้ unlock A
 - ถ้าผิดกฎ behavior จะเป็นแบบ Undefined และขึ้นอยู่กับ Implementation ซึ่งบังเอิญ glibc version ที่ ubuntu ใช้ปัจจุบันไม่เช็คและให้ทำได้

ตัวแปร Mutex variable

- เป็นตัวแทนของการทำ lock/unlock synchronization
- มีชนิดเป็น `pthread_mutex_t` data type.
- เราสามารถกำหนดค่าเริ่มต้นให้เป็น `PTHREAD_MUTEX_INITIALIZER` (สำหรับตัวแปรที่ประกาศในโปรแกรม) หรือ
- กำหนดค่าเริ่มต้นด้วย `pthread_mutex_init()`
- เมื่อเลิกใช้งานให้เรียก `pthread_mutex_destroy()`

Init and destroy

```
#include <pthread.h>
```

```
int pthread_mutex_init(pthread_mutex_t *restrict mutex,  
                        const pthread_mutexattr_t *restrict attr);
```

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Both return: 0 if OK, error number on failure

- Attr is set to NULL

Lock/Trylock/Unlock

```
#include <pthread.h>
```

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

All return: 0 if OK, error number on failure

mutexex1.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t mutex_var;
7 int global_var = 100;
```

mutexex1.c

```
8
9 void * foo(void * arg){
10
11     pthread_mutex_lock(&mutex_var);
12     global_var += 25;
13     printf("foo() update global_var to %d\n", global_var);
14     pthread_mutex_unlock(&mutex_var);
15
16     pthread_exit((void *)NULL);
17 }
```

mutexex1.c

```
18
19 int main(void){
20
21     pthread_t mtid, ntid;
22     void *tret;
23
24     pthread_mutex_init(&mutex_var, NULL);
25
26     pthread_create(&ntid, NULL, foo, NULL);
27
28     pthread_mutex_lock(&mutex_var);
29     global_var += 75;
30     printf("main() update global_var to %d\n", global_var);
31     pthread_mutex_unlock(&mutex_var);
32
33     pthread_join(ntid, tret);
34
35 }
```

Mutex

	Main thread	Foo thread
	Lock(&mutex_var)	Lock(&mutex_var)
	Global_var += 75;	
	Printf("...."); // 175	
	Unlock(&mutex_var)	
		Global_var += 25;
		Printf("...."); // 200
		Unlock(&mutex_var)

Critical section

- ส่วนของโปรแกรมที่อยู่ภายใต้การ synchronization ด้วย mutex lock และ unlock operations เรียกว่า **critical section**
- มีเพียง thread เดียวเท่านั้นที่จะเข้าใช้ critical section ของตนได้

t1	t2	t3
Lock(mutexA)	Lock(mutexA)	Lock(mutexA)
Critical section ของ t1		
Unlock(mutexA)		
	Critical section ของ t2	
	Unlock(mutexA)	
		Critical section ของ t3
		Unlock(mutexA)

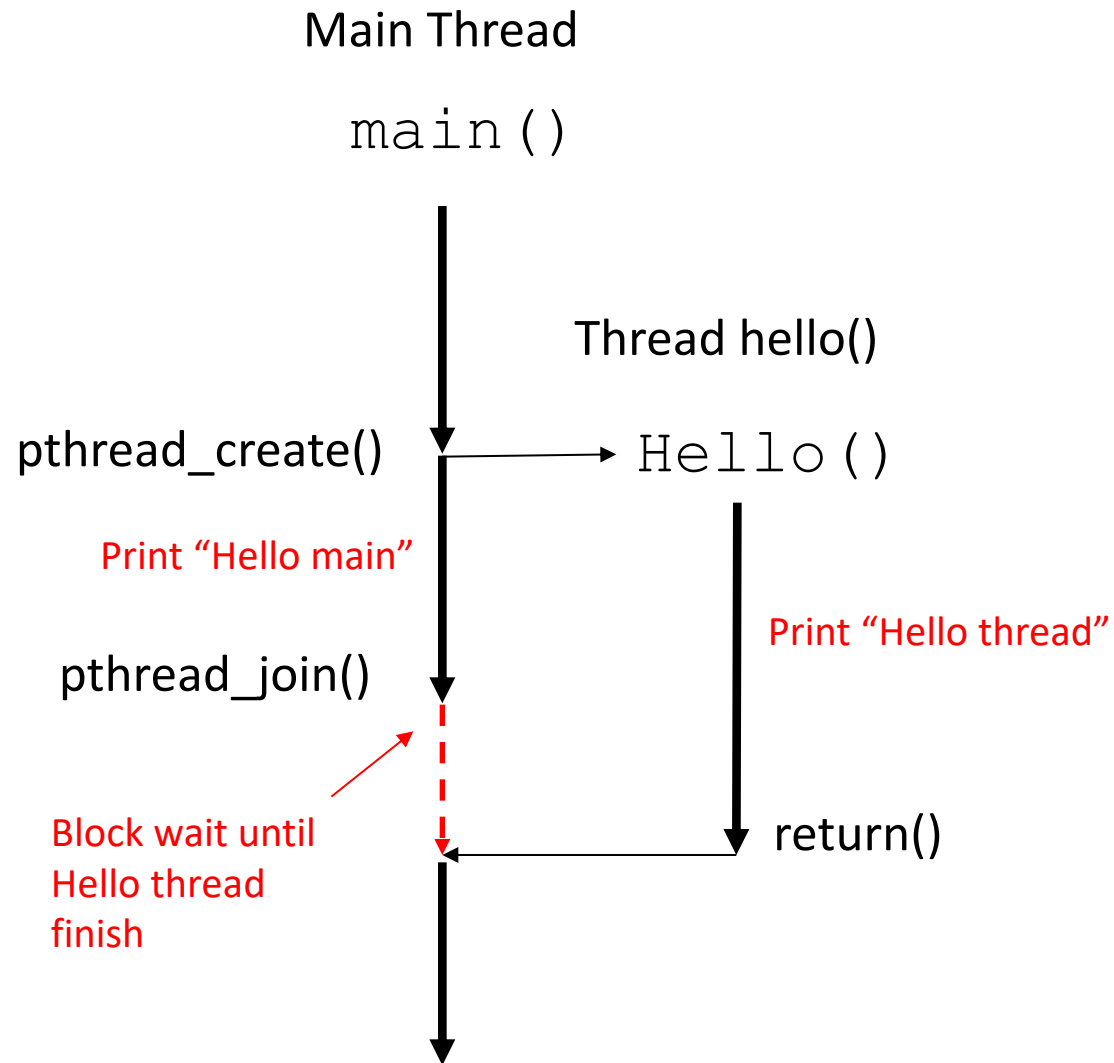
Critical section

- ส่วนใหญ่ code ใน critical section จะเป็นการเข้าถึง shared resource เช่น global variables หรือ data structures ใน heap segment e.g. `int Global_var = 0;`

t1	t2	t3
Lock(mutexA)	Lock(mutexA)	Lock(mutexA)
Global_var++		
Unlock(mutexA)		
	Global_var++	
	Unlock(mutexA)	
		Global_var++
		Unlock(mutexA)

Critical section

- Critical section เป็นปฏิปักษ์กับ parallelism
- ในการเขียนโปรแกรม multithread คุณต้องการให้หลายๆ thread ช่วยกันทำงาน
- แต่การใช้ mutex synchronization เป็นการบังคับให้ thread จำนวนหนึ่งใน process ที่ต้องการ lock ตัวแปร mutex ตัวเดียวกัน **หยุดการทำงานแบบ parallel แล้วประมวลผล critical section ของแต่ละ thread ได้ที่ละ thread เท่านั้น**



จำลอง pthread_join() ด้วย mutex

- Pthread_join() เป็นการ synchronization ระหว่าง 2 threads
- Main thread จะรอจนกระทั่ง thread ที่รอจบการทำงาน
- **ทำอะไรจึงจะทำให้ main thread รอ และ**
 - การรอด้วย mutex เกิดขึ้นเมื่อ mutex ถูก lock ไปแล้วครั้งหนึ่งและจะรอการ lock ครั้งที่สอง
- **ทำอะไรเมื่อ thread ที่ถูกรอจบการทำงานแล้วทำอะไรบางอย่างเพื่อให้ main thread หยุดรอและกลับไปทำงานคำสั่งถัดไป**
 - การหยุดรอของ thread ใดๆก็ตามที่รอ lock mutex variable หนึ่งอยู่จะเกิดขึ้น เมื่อมี thread อื่น unlock mutex variable นั้น

จำลอง join

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t mutex_var;
7
8 void * foo(void * arg){
9     pthread_t tid = pthread_self();
10    printf("foo tid = %lu do something\n", (unsigned long)tid);
11
12    sleep(5);
13
14    printf("foo unlock mutex\n");
15    pthread_mutex_unlock(&mutex_var);
16    pthread_exit((void *)NULL);
17 }
18
```

จำลอง join

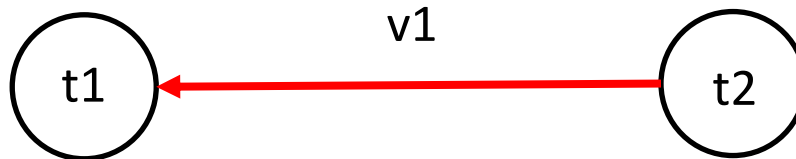
```
19 int main(void){
20
21     pthread_t ntid;
22     void *tret;
23
24     pthread_mutex_init(&mutex_var, NULL);
25
26     pthread_create(&ntid, NULL, foo, NULL);
27     pthread_detach(ntid);
28     printf("main thread do something \n");
29
30     //pthread_join(ntid, tret);
31     pthread_mutex_lock(&mutex_var);
32     printf("main thread wait here...\n");
33     pthread_mutex_lock(&mutex_var);
34     printf("main thread enters critical section and leave...\n");
35     pthread_mutex_unlock(&mutex_var);
36
37 }
```

ข้อผิดพลาดของการจำลอง Pthread_join()

1. โปรแกรมอาจเกิด **race condition** ขึ้นได้ ลองพิจารณาดูว่าถ้า Thread foo() รัน pthread_mutex_unlock() ในบรรทัดที่ 15 ก่อน Main Thread รันคำสั่ง pthread_mutex_lock() ในบรรทัดที่ 31 (ต้องแก้ไขอย่างไรจึงจะไม่เกิด race condition จะเป็นการบ้าน)
2. การใช้ pthread_mutex_unlock เพื่อ unlock **mutex variable** ที่ไม่ได้ถูกล็อกด้วยเทรดเดียวกัน ไม่ตรงตาม POSIX standard ผลลัพธ์จึงเป็นแบบ Undefined และขึ้นอยู่กับ Implementation
 1. https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_mutex_unlock.html
3. บั๊กใน glibc library ที่ใช้บน ubuntu ไม่ได้ใช้คำว่า thread ที่ lock และ unlock mutex ต้องเป็นเทรดเดียวกัน code ข้างต้นจึงทำได้
 1. Implementation ที่ถูกต้อง ต้องใช้ semaphore แทน (การบ้าน? May be?)

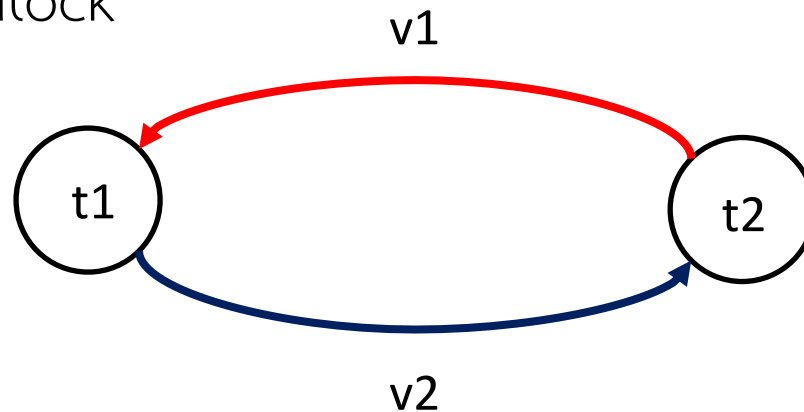
deadlock

- Deadlock คือสถานะการณ์ที่มีการ รอ กันไปมาจนหาทางออกไม่ได้
- เกิด graph แห่งการรอที่เป็น วงกลม
- กำหนดให้ เมื่อ thread t2 รอ mutex v1 ที่ thread t1 ได้ไปให้เขียนเป็น



deadlock

- Deadlock ระหว่าง thread t1 และ t2 จะเกิดขึ้นเมื่อ t2 รอ mutex v1 ที่ t1 ได้ไปและในขณะเดียวกัน t1 ก็รอ mutex v2 ที่ t2 ได้ไป ดังภาพ
- Circular wait ของการรอ อาจซับซ้อนกว่านี้เมื่อใช้ mutex หลายตัวแปร ทำให้เกิด graph แห่งการรอคอยขนาดใหญ่ แต่เมื่อใดที่มีวงกลมใน graph นั้นก็เกิด deadlock



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t v1, v2;
7
8 void * t(void * arg){
9     long int id = (long) arg;
10    if(id == 1){
11        pthread_mutex_lock(&v1);
12        sleep(2);
13        pthread_mutex_lock(&v2);
14        printf("enter critical section of t(1)\n");
15        pthread_mutex_unlock(&v2);
16        pthread_mutex_unlock(&v1);
17    }
18    else if(id == 2){
19        pthread_mutex_lock(&v2);
20        sleep(2);
21        pthread_mutex_lock(&v1);
22        printf("enter critical section of t(2)\n");
23        pthread_mutex_unlock(&v1);
24        pthread_mutex_unlock(&v2);
25    }
26    pthread_exit((void *)NULL);
27 }
```

```
29 int main(void){
30
31     pthread_t ntid1, ntid2;
32
33     pthread_mutex_init(&v1, NULL);
34     pthread_mutex_init(&v2, NULL);
35
36     pthread_create(&ntid1, NULL, t, (void *)1);
37     pthread_create(&ntid2, NULL, t, (void *)2);
38
39     pthread_join(ntid1, NULL);
40     pthread_join(ntid2, NULL);
41
42 }
```

deadlock

- เกิด Deadlock

```
openstack@source:~/system-programming/pthread$ gcc -o e4 mutexex4-deadlock1.c -lpthread
openstack@source:~/system-programming/pthread$ ./e4
```

- ขอให้ดู code ถัดไป ถ้าเข้าไป modify ให้โปรแกรมพิมพ์ข้อมูลบนทั้งสอง
เทอร์มินัลโปรแกรมอีกครั้ง

```
9 void * t(void * arg){
10     long int id = (long) arg;
11     pthread_t tid = pthread_self();
12     printf("this is tr%ld tid = %lu \n", id, (unsigned long)tid);
13
14     if(id == 1){
15         pthread_mutex_lock(&v1);
16         pthread_mutex_lock(&v2);
17         printf("enter critical section of t(1)\n");
18         pthread_mutex_unlock(&v2);
19         pthread_mutex_unlock(&v1);
20     }
21     else if(id == 2){
22         pthread_mutex_lock(&v2);
23         pthread_mutex_lock(&v1);
24         printf("enter critical section of t(2)\n");
25         pthread_mutex_unlock(&v1);
26         pthread_mutex_unlock(&v2);
27     }
28     pthread_exit((void *)NULL);
29 }
```

deadlock

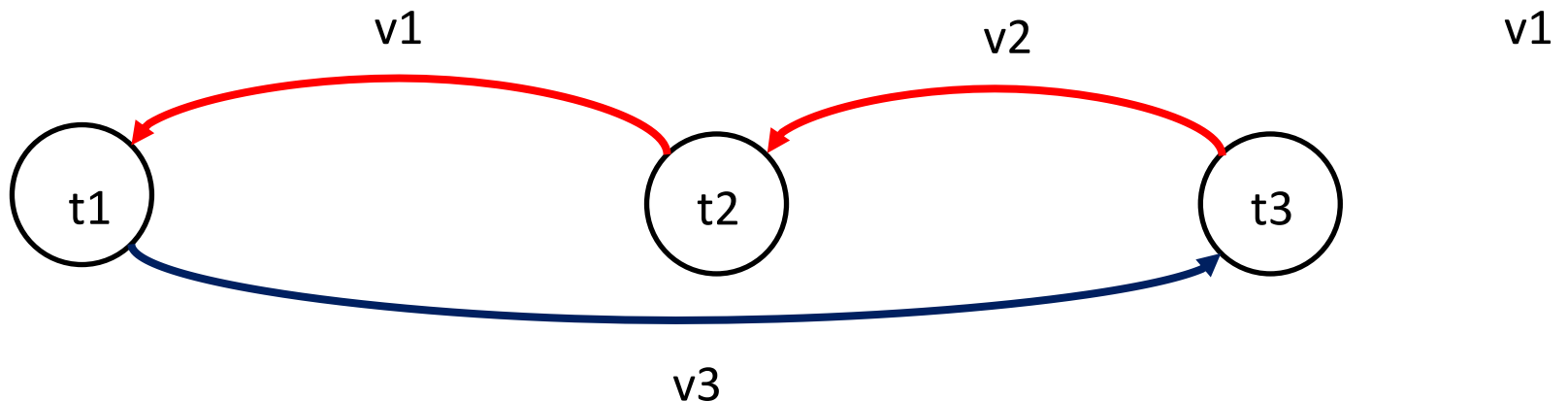
- ผลคือไม่เกิด Deadlock

```
openstack@source:~/system-programming/pthread$ gcc -o e5 mutexex5-deadlock2.c -lpthread
openstack@source:~/system-programming/pthread$ ./e5
this is tr1 tid = 140608515835648
enter critical section of t(1)
this is tr2 tid = 140608507442944
enter critical section of t(2)
openstack@source:~/system-programming/pthread$
```

- เพราะเหตุใด
- ใครเป็นผู้จัดจ้หวะการทำงานของแต่ละ thread
- ปัญหาในการเขียนโปรแกรมที่ตรวจจับได้ยากที่สุดก็คือปัญหาที่บางครั้งโปรแกรมทำงานถูกบางครั้งก็ทำงานผิดขึ้นอยู่กับสถานะแวดล้อมที่มีความไม่แน่นอน ทำให้โปรแกรมไม่มีความน่าเชื่อถือ (unreliable)
- เราต้องทำให้โปรแกรมประมวลผลอย่างเชื่อถือได้ (reliable)

Circular Waits

- Lock(v1)
- Lock(v2)
- Lock(v3)
- Lock(v3)
- Lock(v1)
- Lock(v2)



ป้องกันการเกิด deadlock (deadlock prevention)

- การแก้ปัญหา deadlock ทำได้โดยการสร้างแนวทางปฏิบัติเวลาที่โปรแกรมมีการใช้ตัวแปร mutex หลายๆ ตัวแปรในโปรแกรม
- ขอให้สังเกตว่าปัญหาเกิดขึ้นเมื่อต่างเทรดยึดตัวแปรบางตัวไว้
- t1 ยึด v1 และต้องการ v2 และ t2 ยึด v2 ต้องการ t1
- เกิดจากอันดับในการ lock ที่โปรแกรมเมอร์เขียนให้เป็นเช่นนั้น
- วิธีการแก้คือการจัดอันดับให้ไม่เกิดสถานะการัน ยึดบางตัวขึ้น
- โดยใช้ “ข้อควรปฏิบัติในการเขียน code” ที่ใช้กับทุกเทรดเหมือนกัน

```
8 void * t(void * arg){
9     long int id = (long) arg;
10    if(id == 1){
11        pthread_mutex_lock(&v1);
12        sleep(2);
13        pthread_mutex_lock(&v2);
14        printf("enter critical section of t(1)\n");
15        pthread_mutex_unlock(&v2);
16        pthread_mutex_unlock(&v1);
17    }
18    else if(id == 2){
19        pthread_mutex_lock(&v1);
20        sleep(2);
21        pthread_mutex_lock(&v2);
22        printf("enter critical section of t(2)\n");
23        pthread_mutex_unlock(&v2);
24        pthread_mutex_unlock(&v1);
25    }
26    pthread_exit((void *)NULL);
27 }
```

แก้ปัญหา deadlock

- จากภาพโปรแกรมเมอร์แก้ปัญหาโดย
- ในกรณีที่โปรแกรมมีการเรียกใช้ตัวแปร mutex หลายตัวแปรได้แก่ v1 และ v2 ถ้าเราจัดอันดับการเรียกใช้ตัวแปร mutex ของทุกเทร็ดให้เป็นในอันดับเดียวกัน ก็จะแก้ปัญหา deadlock ได้
- จากตัวอย่างก่อนหน้านี้ t(1) และ t(2) ต้องการ lock v1 ก่อนแล้วจึง lock v2 เหมือนกัน
- ดังนั้นถ้าเทร็ดใดๆในโปรแกรมต้องการ lock mutex แบบซ้อนๆกัน (nested) ก็ให้ทำตามอันดับ v1 v2 v3 ... vn ทุกเทร็ด (บางเทร็ดอาจไม่ได้ใช้หมดทุก v แต่ขอให้ทำตามอันดับ ห้ามสลับอันดับ) ก็จะปิด deadlock possibility ได้

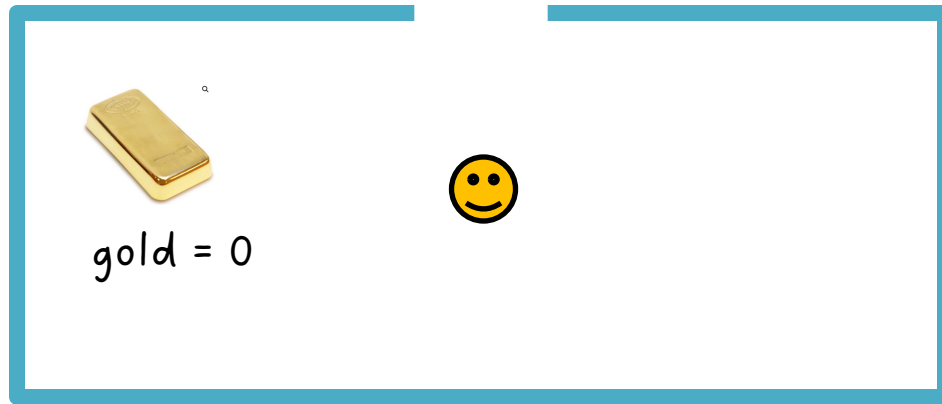
Conditional Variables

- ในการประสานงานด้วย mutex อาจมีสถานการณ์ที่ เทรดหนึ่งได้ lock mutex แล้ว เมื่อทำงานอยู่ใน critical section พบว่าไม่สามารถทำงานต่อได้เพราะเงื่อนไขบางอย่างไม่เป็นไปตามที่ต้องการ
- เทรดนั้นจึง ต้องการ block หยุดรอให้เงื่อนไขนั้นผ่านก่อน และ
- Unlock mutex นั้นเพื่อให้เทรดอื่นได้ lock mutex เพื่อทำงานอื่นได้
- หลังจากที่เทรดอื่นได้ใช้งาน mutex และเปลี่ยนเงื่อนไขที่เทรดแรก block รออยู่ ให้เป็นไปตามที่เทรดแรกต้องการแล้ว เทรดอื่นนั้นก็ส่ง signal มาปลุกให้เทรดแรกให้ตื่นการ block เพื่อให้เทรดแรกได้เข้าไปแย่ง lock mutex เพื่อที่จะได้ทำงานใน critical section ต่อไป



gold = 0





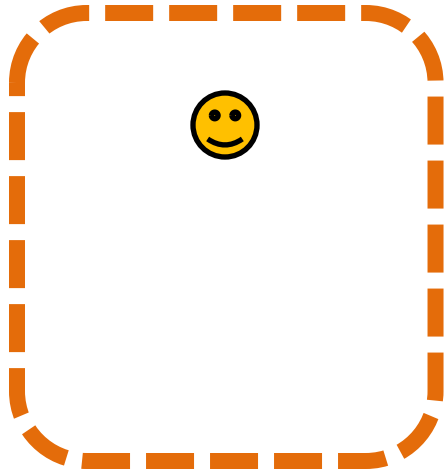
gold = 0





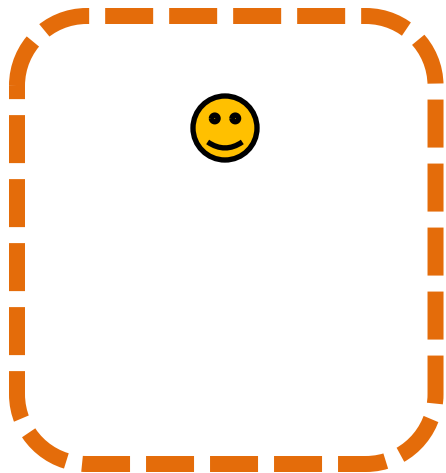
gold = 0





gold = 0



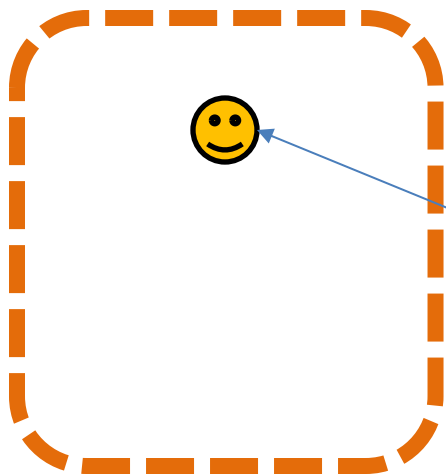


gold = 1



ทำงานใน critical
Section คือ gold++



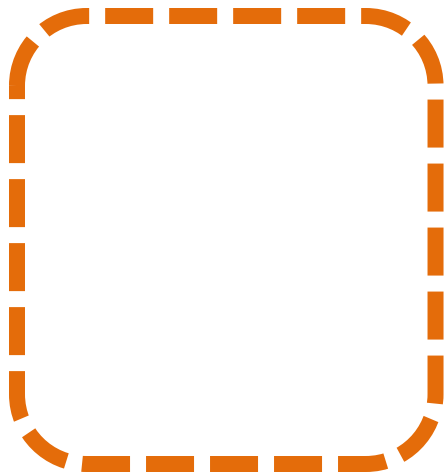


gold = 1



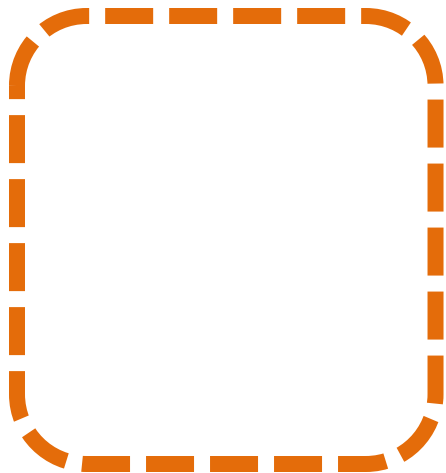
Signal ปลุกคนใดคนหนึ่งในห้องรอ
ว่ามีทองแล้วนะ





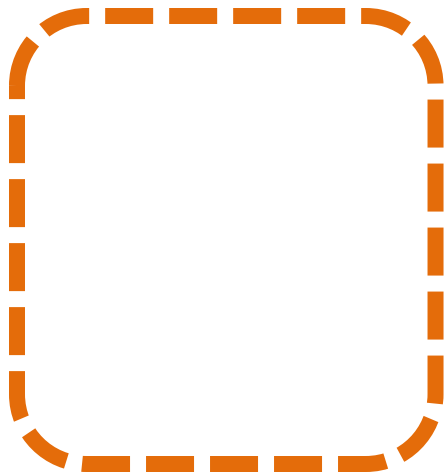
gold = 1





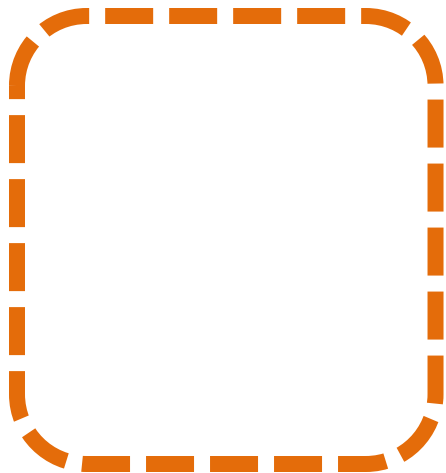
gold = 1





gold = 1



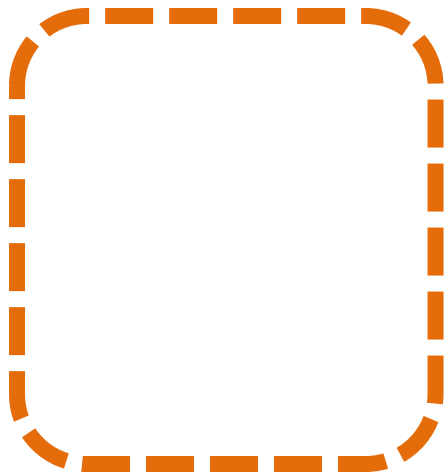


gold = 0



ทำงานใน **critical**
Section คือผลิตสร้อยและ
gold--





gold = 0

necklace = 1





gold = 0





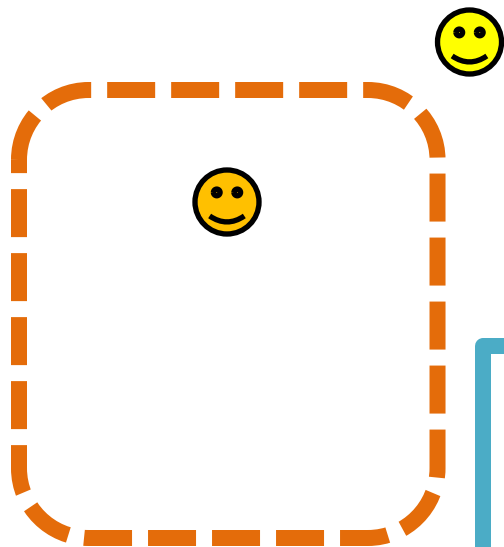
gold = 0



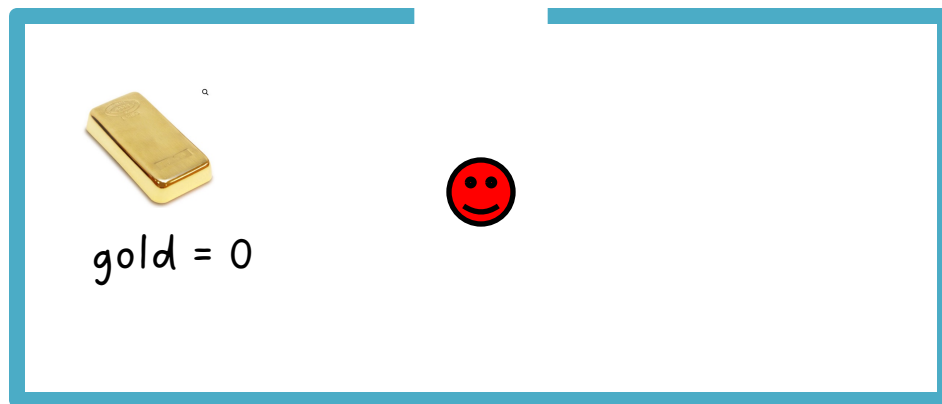


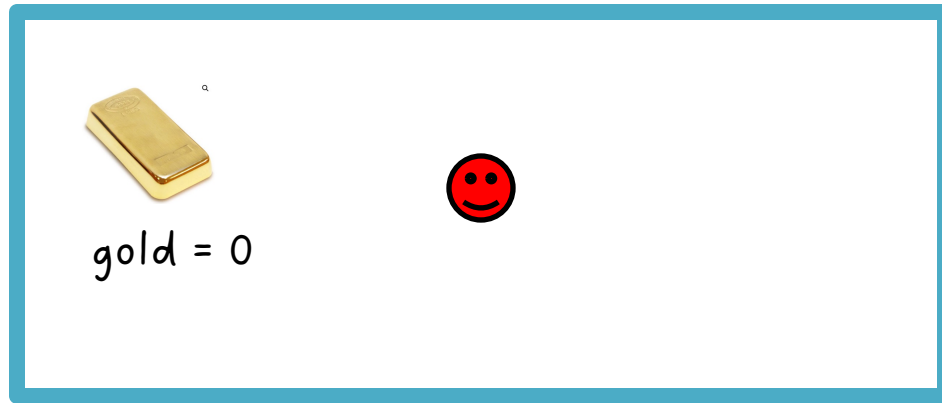
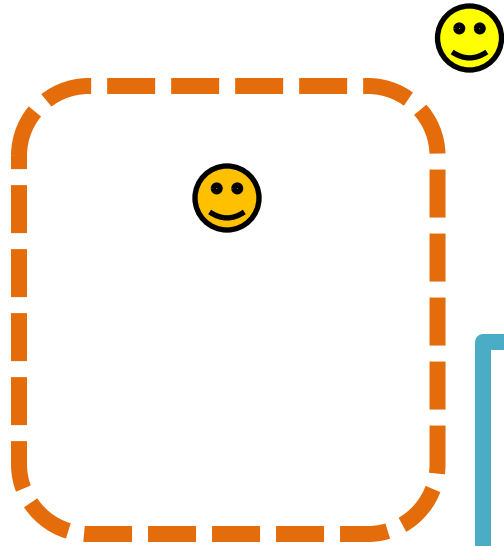
gold = 0

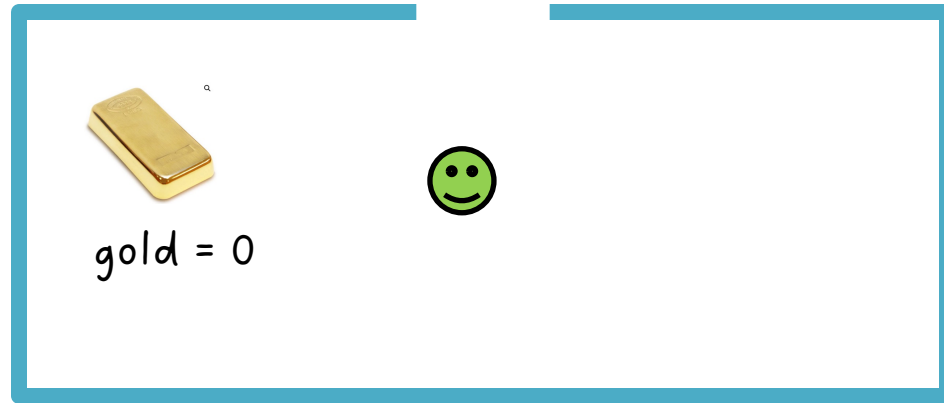
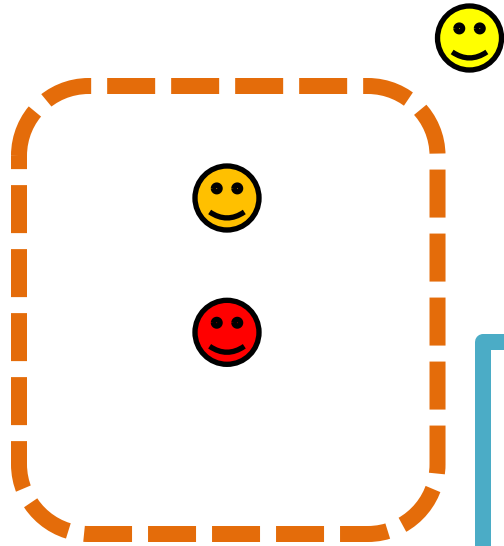


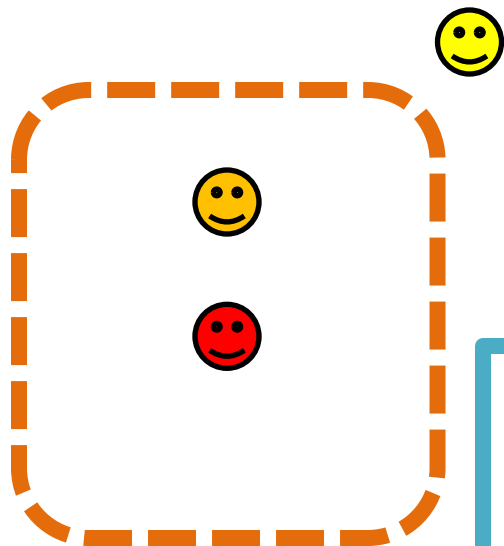


๗
ห้องนอนรอ







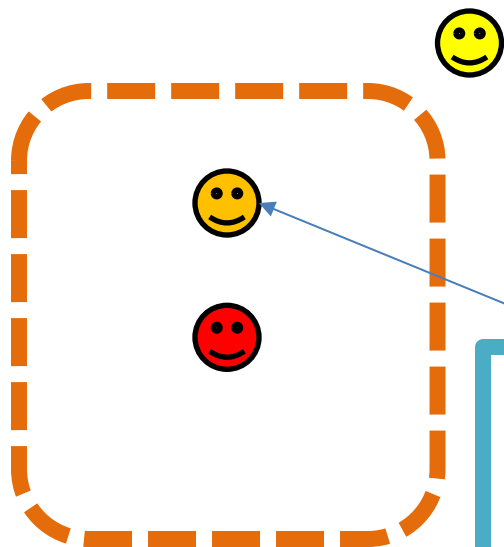


gold = 1



ทำงานใน critical
Section คือ gold++



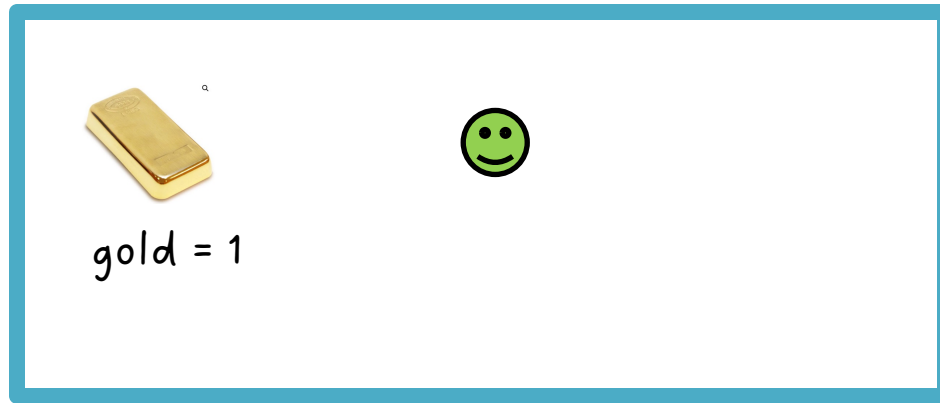
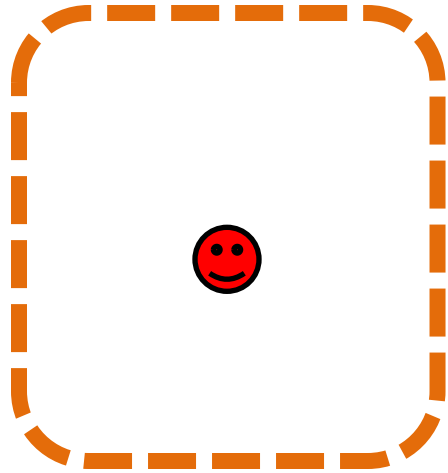


gold = 1



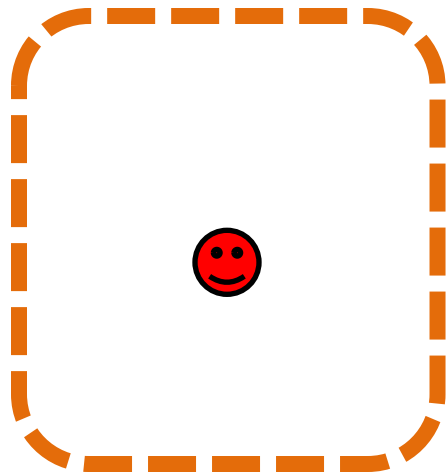
Signal ปลุกคนใดคนหนึ่งในห้องรอ
ว่ามีทองแล้วนะ





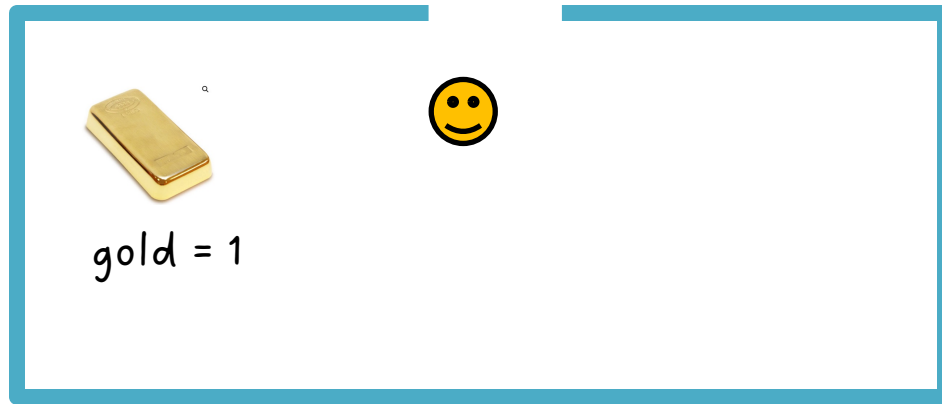
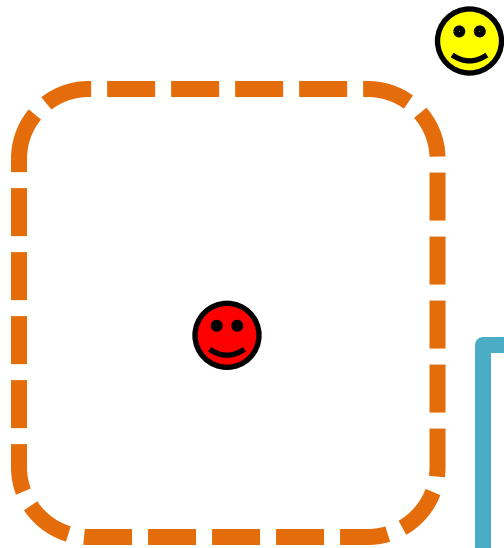
gold = 1

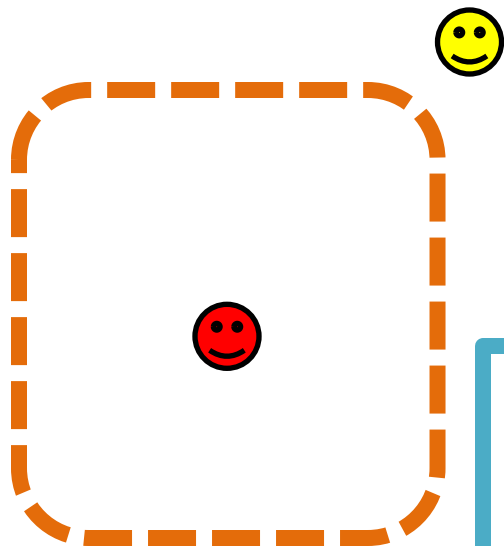




gold = 1





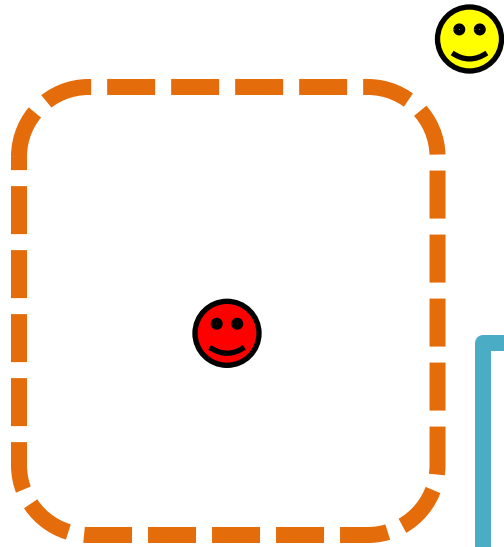


gold = 0



ทำงานใน **critical**
Section คือผลิตสร้อยและ
gold--







gold = 0





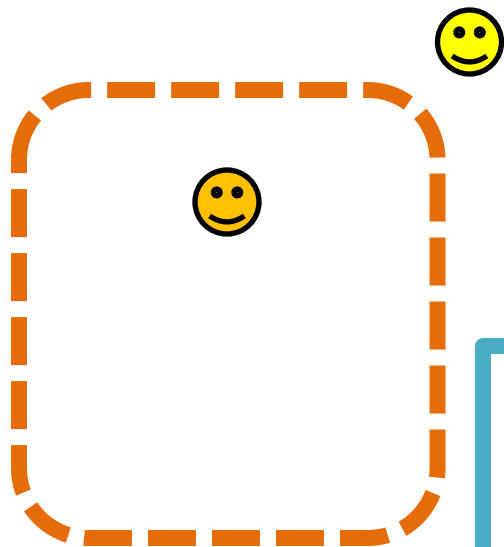
gold = 0



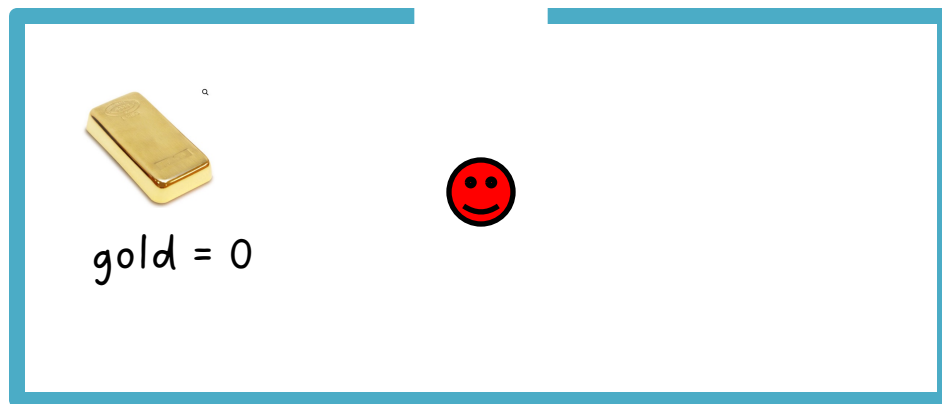


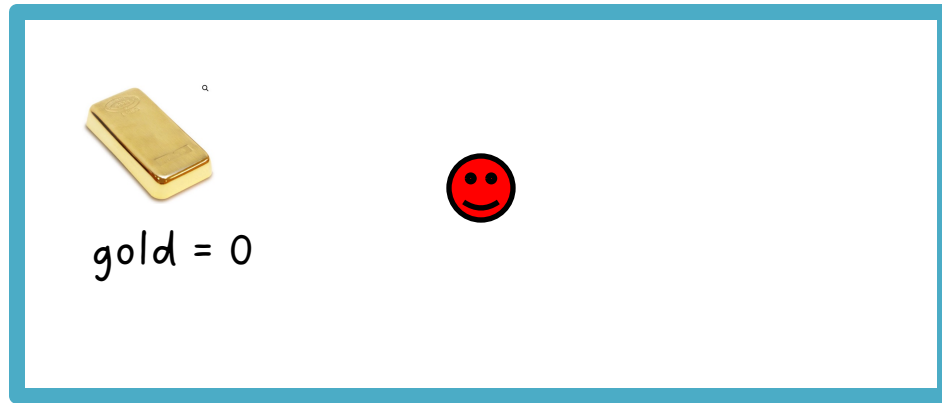
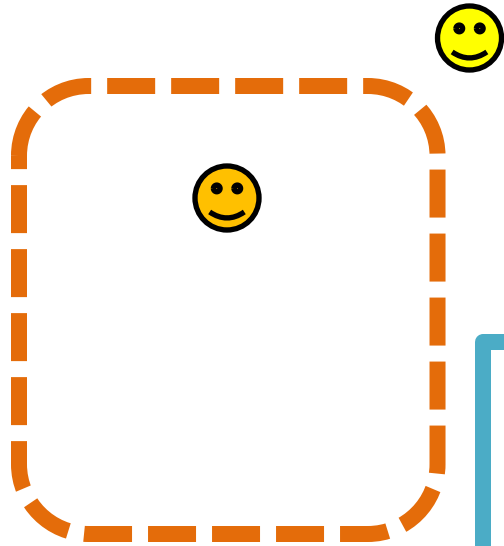
gold = 0

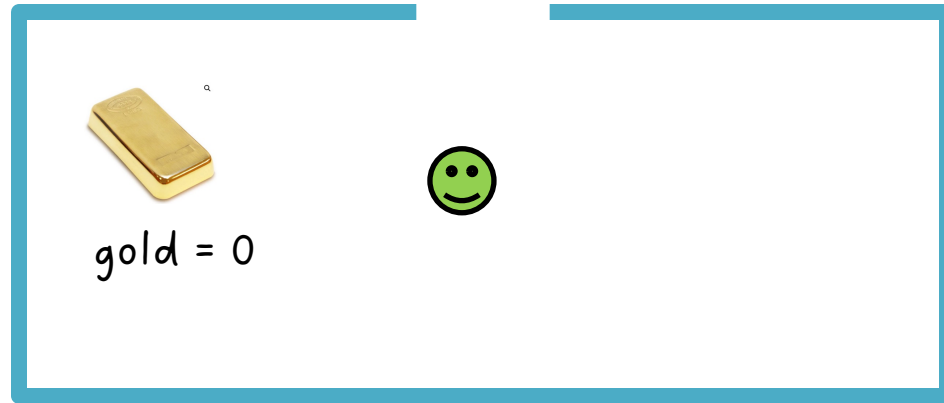
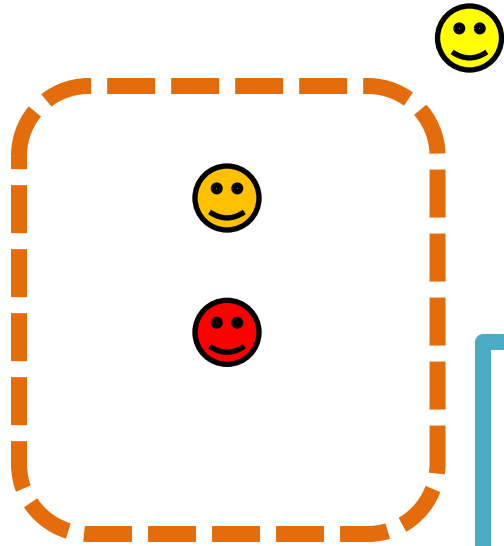


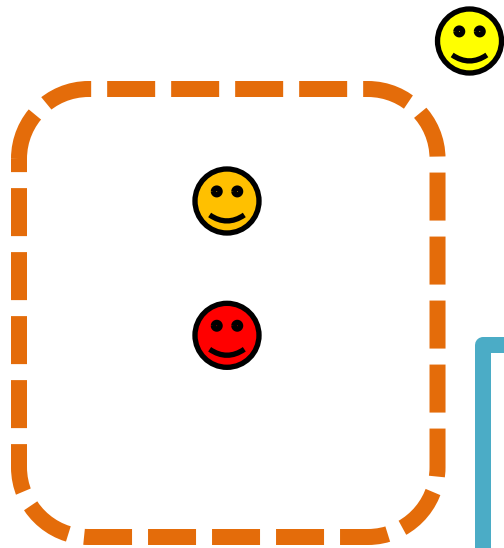


๗
ห้องนอนรอ







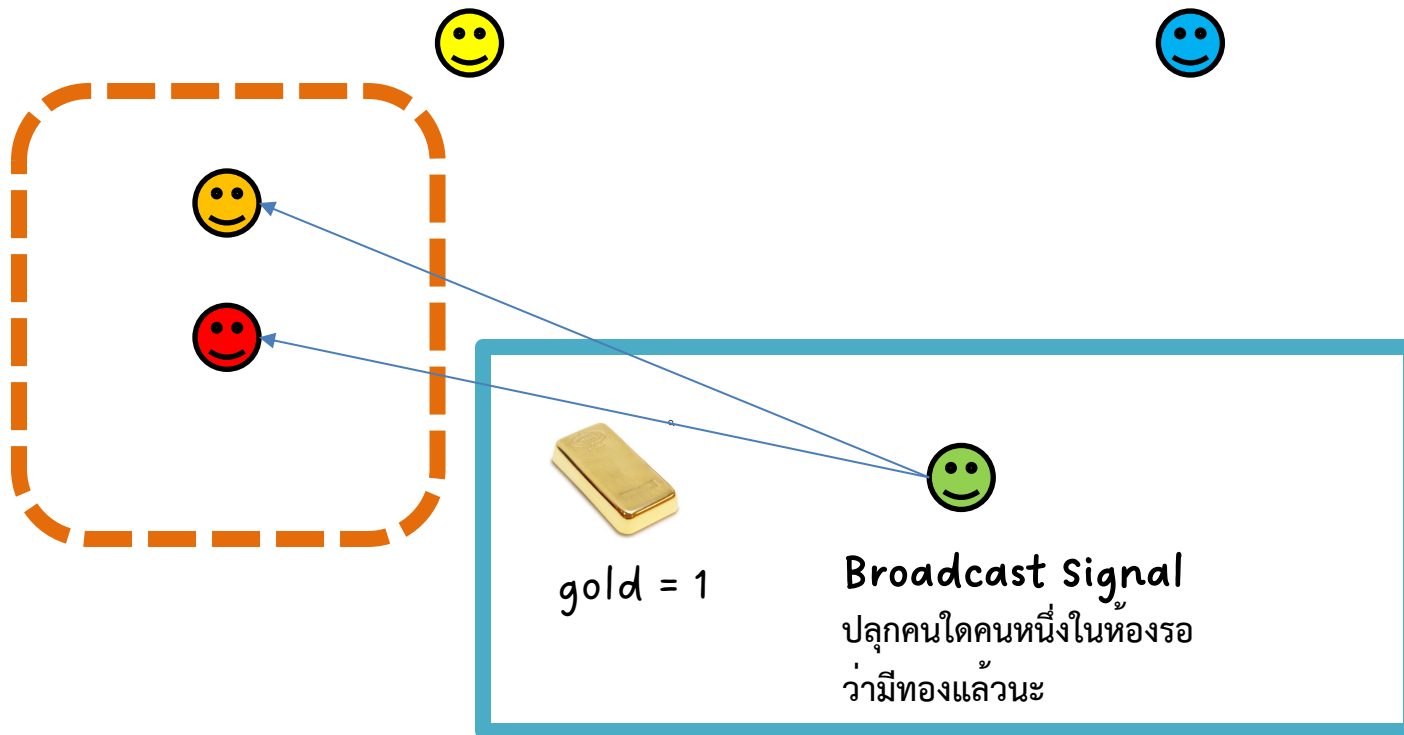


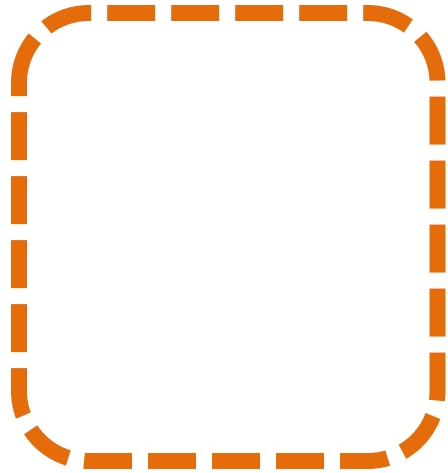
gold = 1



ทำงานใน critical
Section คือ gold++

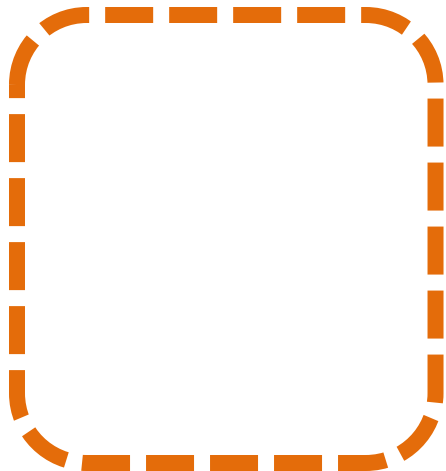






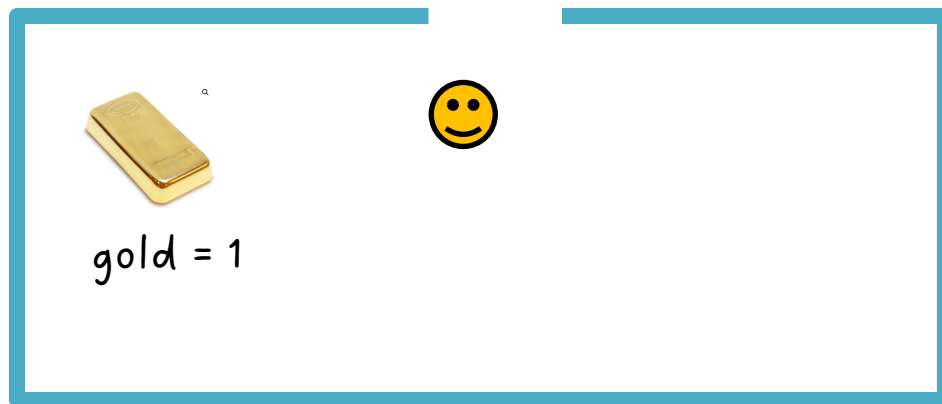
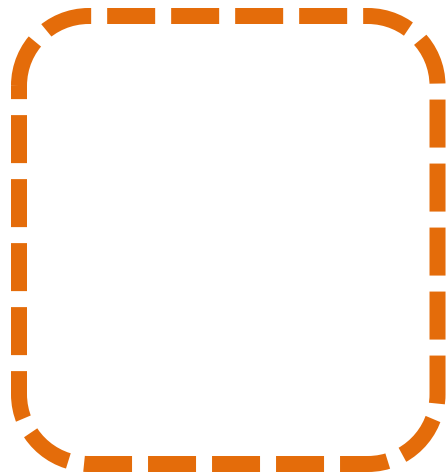
gold = 1





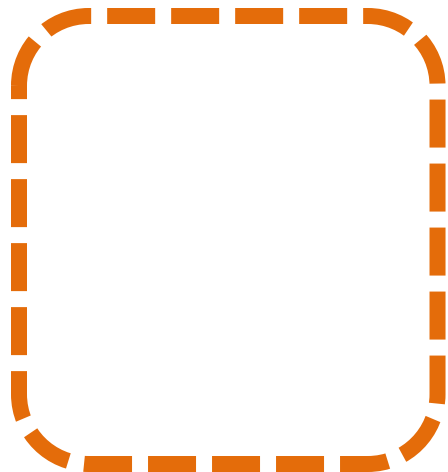
gold = 1





gold = 1



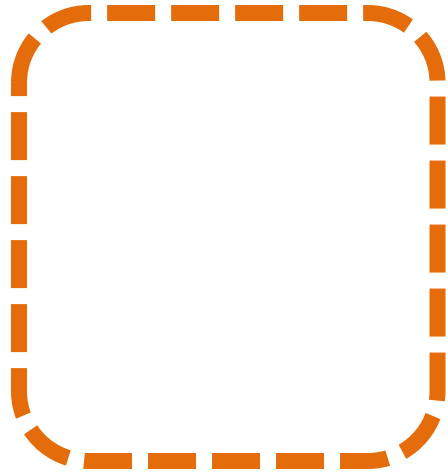


gold = 0



ทำงานใน **critical**
Section คือผลิตสร้อยและ
gold--





gold = 0

necklace = 1



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 pthread_mutex_t mutex_var;
7 pthread_cond_t cv;
8
9 int global_var = 0;
10
```

Mutex Var



Conditional Var



```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n", global_var);
15
16     while(global_var <= 0){
17         pthread_cond_wait(&cv, &mutex_var);
18     }
19     printf("foo() critical 2: new global_var is %d\n", global_var);
20
21     global_var--;
22     printf("foo() critical 2: update global_var to %d\n", global_var);
23
24     pthread_mutex_unlock(&mutex_var);
25
26     pthread_exit((void *)NULL);
27 }
```

Lock(mutex)

```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n", global_var);
15     while(global_var <= 0){
16         pthread_cond_wait(&cv, &mutex_var);
17     }
18     printf("foo() critical 2: new global_var is %d\n", global_var);
19     global_var--;
20     printf("foo() critical 2: update global_var to %d\n", global_var);
21     pthread_mutex_unlock(&mutex_var);
22
23     pthread_exit((void *)NULL);
24 }
25
26
27 }
```

Lock(mutex)

Critical 1

```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n"
15           , global_var);
16     while(global_var <= 0){
17         pthread_cond_wait(&cv, &mutex_var);
18     }
19     printf("foo() critical 2: new global_var is %d\n"
20           , global_var);
21     global_var--;
22     printf("foo() critical 2: update global_var to %d\n"
23           , global_var);
24     pthread_mutex_unlock(&mutex_var);
25
26     pthread_exit((void *)NULL);
27 }
```

Lock(mutex)

Condition == true

```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n", global_var);
15
16     while(global_var <= 0){
17         pthread_cond_wait(&cv, &mutex_var);
18     }
19     printf("foo() critical 2: new global_var is %d\n", global_var);
20
21     global_var--;
22     printf("foo() critical 2: update global_var to %d\n", global_var);
23
24     pthread_mutex_unlock(&mutex_var);
25
26     pthread_exit((void *)NULL);
27 }
```

Lock(mutex)

unLock(mutex)
& wait for (cv)

```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n",
15           global_var);
16     while(global_var <= 0){
17         pthread_cond_wait(&cv, &mutex_var);
18     }
19     printf("foo() critical 2: new global_var is %d\n",
20           global_var);
21     global_var--;
22     printf("foo() critical 2: update global_var to %d\n",
23           global_var);
24     pthread_mutex_unlock(&mutex_var);
25
26     pthread_exit((void *)NULL);
27 }
```

Lock(mutex)

Critical 1

Condition == true

unLock(mutex)
& wait for (cv)

Operation on foo() (ตอน 1)

- จาก slide ก่อนหน้า เทรด foo() จะเริ่มทำงานโดย lock(mutex_var) แล้วเข้ามาทำงานใน critical section 1
- หลังจากนั้นมันจะเช็คค่า global_var น้อยกว่าหรือเท่ากับ 0 หรือไม่
- ถ้าใช่ foo() ก็จะรันคำสั่ง pthread_cond_wait(&cv, &mutex_var) ซึ่งเป็น atomic operation ที่ทำสองสิ่งได้แก่
 - หยุดการประมวลผลของเทรด foo และรอจนกว่าจะได้รับแจ้งผ่านตัวแปร condition variable “cv” ว่าให้ตื่นขึ้นเลิกรอได้แล้ว
 - Unlock(mutex_var) เพื่อให้เทรดอื่นเข้าไปทำ critical section ของตนได้

```
29 void * bar(void * arg){
30     sleep(3);
31     pthread_mutex_lock(&mutex_var);
32     global_var ++;
33     printf("foo() update global_var to %d\n", global_var);
34     pthread_cond_signal(&cv);
35     pthread_mutex_unlock(&mutex_var);
36
37     pthread_exit((void *)NULL);
38 }
```

The diagram illustrates the execution flow of the `bar` function. Annotations with arrows point to specific lines of code:

- Lock(mutex)** points to line 31: `pthread_mutex_lock(&mutex_var);`
- Change condition** points to line 32: `global_var ++;`
- unLock(mutex)** points to line 35: `pthread_mutex_unlock(&mutex_var);`

Operation on bar()

- จาก slide ก่อนหน้า สมมติว่าเทรต bar() ทำงานหลังเทรต foo() และรัน lock(mutex_var) หลังจาก foo()
- เทรต bar() จะรออยู่ที่ lock(mutex_var) จนกระทั่ง foo() unlock(mutex_var) ด้วยคำสั่ง pthread_cond_wait()
- หลังจากนั้น bar() จะเข้าสู่ critical section และเพิ่มค่าของ global_var ไป 1 ส่งผลให้ global_var มีค่าเป็น 1 และ
- เรียก pthread_cond_signal(&cv) เพื่อปลุกให้เทรตที่รออยู่ 1 เทรตตื่นขึ้นจากการรอ และ
- Unlock(mutex_var) เพื่อให้เทรตอื่นได้เข้า critical section

```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n"
15           , global_var);
16     while(global_var <= 0){
17         pthread_cond_wait(&cv, &mutex_var);
18     }
19     printf("foo() critical 2: new global_var is %d\n"
20           , global_var);
21     global_var--;
22     printf("foo() critical 2: update global_var to %d\n"
23           , global_var);
24     pthread_mutex_unlock(&mutex_var);
25
26     pthread_exit((void *)NULL);
27 }
```

Got cv;
Lock(mutex)

```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n"
15           , global_var);
16     while(global_var <= 0){
17         pthread_cond_wait(&cv, &mutex_var);
18     }
19     printf("foo() critical 2: new global_var is %d\n"
20           , global_var);
21     global_var--;
22     printf("foo() critical 2: update global_var to %d\n"
23           , global_var);
24     pthread_mutex_unlock(&mutex_var);
25
26     pthread_exit((void *)NULL);
27 }
```

Condition == false

Got cv;
Lock(mutex)

```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n"
15           , global_var);
16     while(global_var <= 0){
17         pthread_cond_wait(&cv, &mutex_var);
18     }
19     printf("critical 2: new global_var is %d\n"
20           , global_var);
21     glob
22     printf("critical 2: update global_var to %d\n"
23           , global_var);
24     pthread_mutex_unlock(&mutex_var);
25
26     pthread_exit((void *)NULL);
27 }
```

Condition == false

Got cv;
Lock(mutex)

Critical 2

```
11 void * foo(void * arg){
12
13     pthread_mutex_lock(&mutex_var);
14     printf("foo() critical 1: block on global_var == %d\n"
15           , global_var);
16     while(global_var <= 0){
17         pthread_cond_wait(&cv, &mutex_var);
18     }
19     printf("foo() critical 2: new global_var is %d\n"
20           , global_var);
21     global_var--;
22     printf("foo() critical 2: update global_var to %d\n"
23           , global_var);
24     pthread_mutex_unlock(&mutex_var);
25
26     pthread_exit((void *)NULL);
27 }
```

Condition == false

Got cv;
Lock(mutex)

unLock(mutex)

Operation on foo() (ตอน 2)

- จาก slide ก่อนหน้า เนื่องจากในโปรแกรมนี้มีเทรด foo() เทรดเดียวที่รอการปลุกผ่านตัวแปร condition variable “cv” ดังนั้น foo() จึงถูกปลุกให้ตื่นขึ้น
 - ในกรณีที่มียเทรดรอ “cv” มากกว่าหนึ่งเทรด คำสั่ง `pthread_cond_signal(&cv)` จะส่งผลให้ระบบ pthread เลือกปลุกเพียงเทรดเดียวเท่านั้น
- หลังจากตื่นแล้ว เทรด foo() จะรีเทรินจากคำสั่ง `pthread_cond_wait()` เพื่อร้นคำสั่งถัดไปได้ก็ต่อเมื่อ เทรด foo() แยก lock(mutex_var) จากเทรดอื่น (ถ้ามี) ได้

Operation on foo() (ตอน 2)

- เนื่องจากในโปรแกรมนี้ไม่มีเธรดอื่นใด เธรด foo() ก็จะแย่ง lock(mutex_var) ได้และเข้าสู่ Critical section 2 ดังภาพ
- ใน critical section 2 ก่อนที่ foo() จะสามารถประมวลผลคำสั่งหลังจาก while loop ได้ เธรด foo() ต้องวน loop มาเช็คเงื่อนไขก่อนว่า global_var ยังมีค่าน้อยกว่าหรือเท่ากับ 0 อยู่หรือไม่
- ในกรณีไม่ เธรด foo() ก็จะจบ while loop และประมวลผลใน critical section 2 ซึ่งจะลดค่า global_var ไป 1 และ unlock(mutex_var) ในที่สุด

Operation on foo() (ตอน 2)

- สาเหตุที่เราใช้ while loop นั้นเนื่องจาก หลังจากที่ foo() ถูกปลุกและกำลังพยายามแย่ง lock(mutex_var) กับเทรตอื่น (ยังไม่มีเทรตอื่นจาก pthread_cond_wait())
- เทรต foo() อาจแพ้ ไม่ได้ lock และมีเทรตอื่นที่ได้ lock(mutex_var) และเข้าไปเปลี่ยนแปลงค่าของ global_var จากเดิมที่ bar() ได้ set ค่าไว้ก่อนที่จะปลุก foo()
- ค่าที่เปลี่ยนไปเช่นนั้นอาจไม่ใช่ค่าที่ critical section 2 ของ foo() ต้องการ
- เพื่อป้องกันสถานะการณ์นี้ foo() จึงต้องวน loop มาเช็ค หลังจากที่ได้ lock ว่าค่าของตัวแปรที่มันจะใช้เป็นจริงตามที่ต้องการหรือไม่ ถ้าจริงก็ทำงานต่อ ถ้าไม่ ก็ต้องเรียก pthread_cond_wait() เพื่อรอ “cv” และ unlock(mutex_var) อีกครั้งหนึ่ง



gold = 0





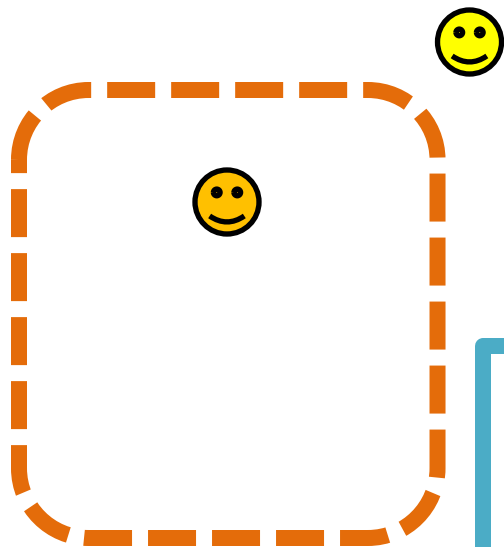
gold = 0



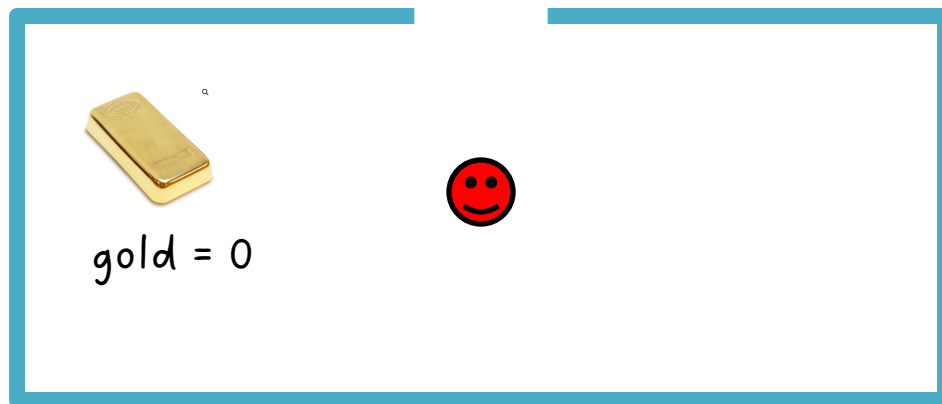


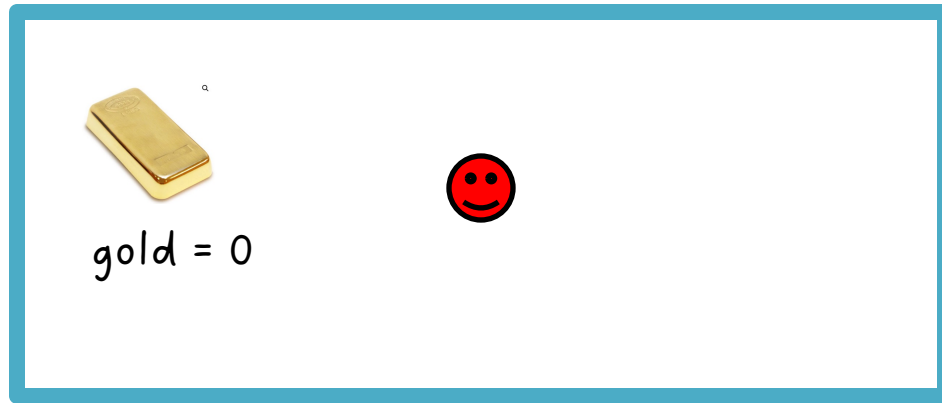
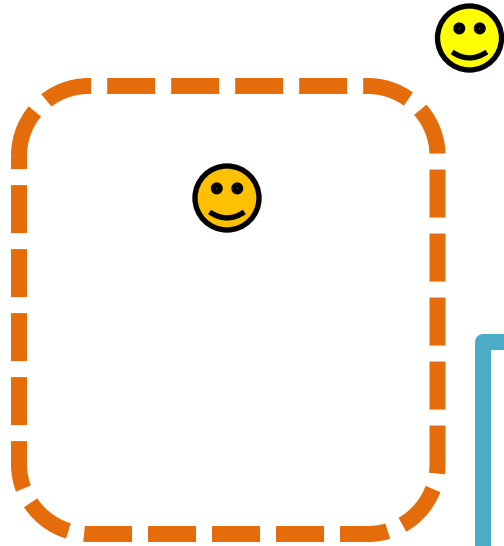
gold = 0

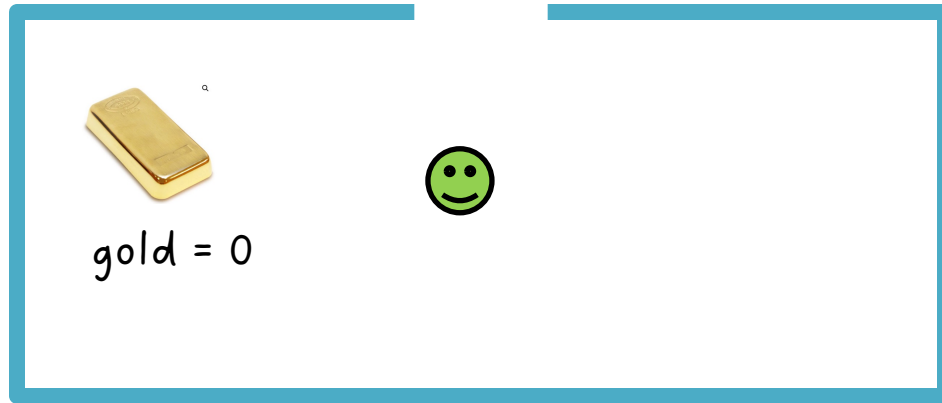
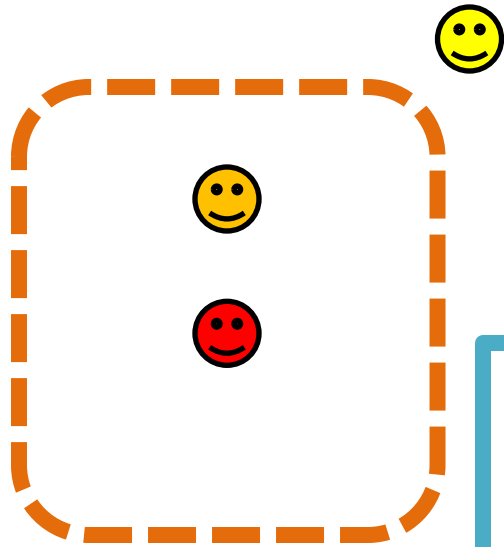


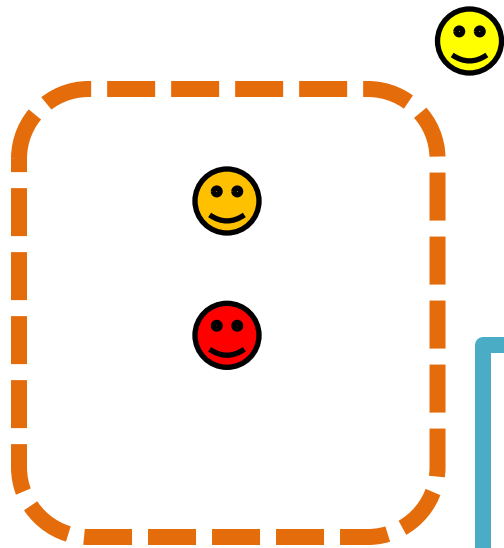


๗
ห้องนอนรอ







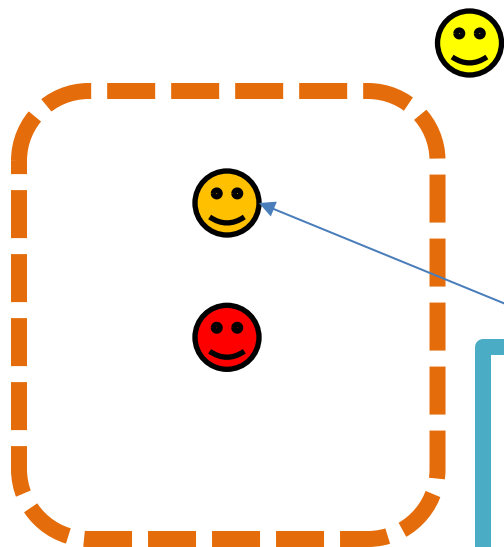


gold = 1



ทำงานใน critical
section คือ gold++



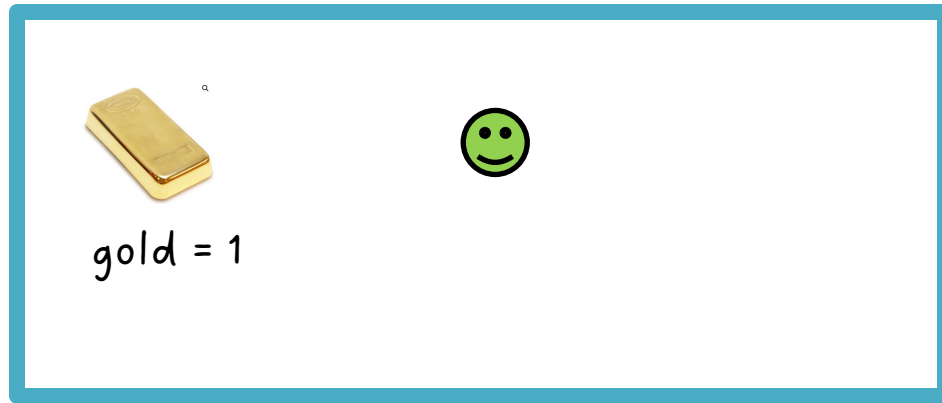
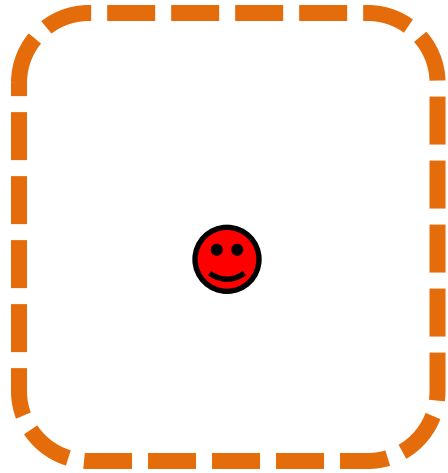


gold = 1



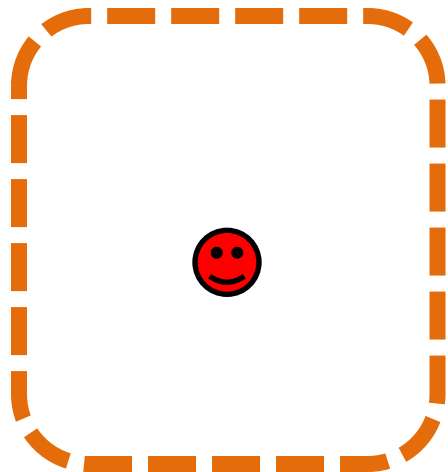
Signal ปลุกคนใดคนหนึ่งในห้องรอ
ว่ามีทองแล้วนะ





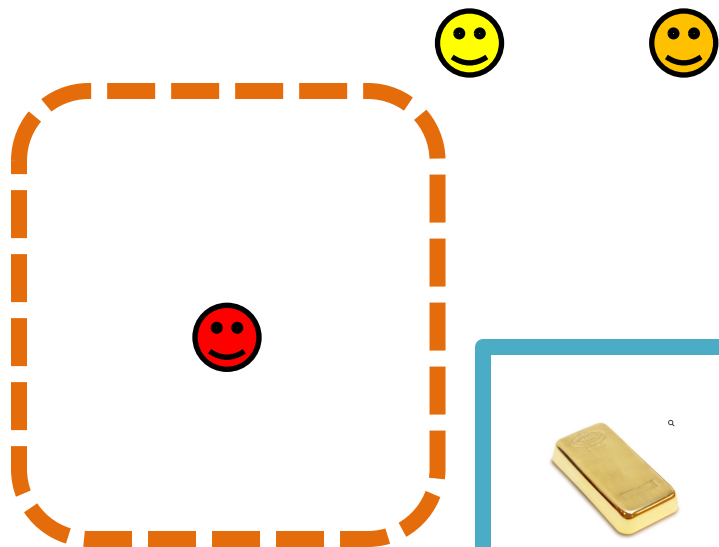
gold = 1





gold = 1



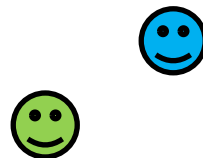
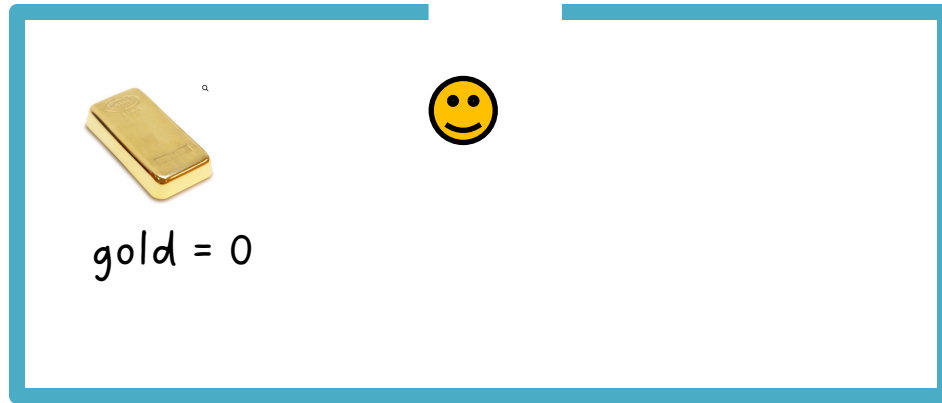
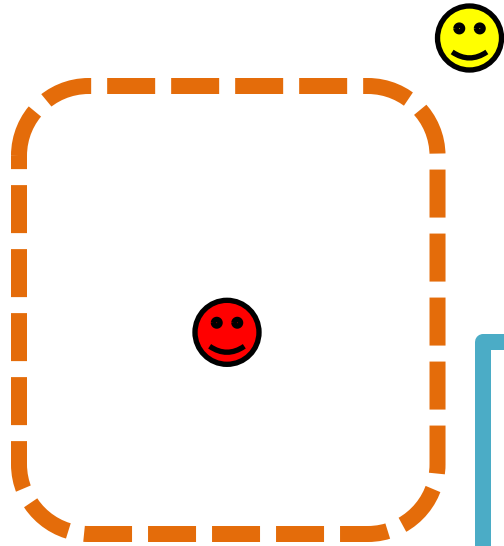


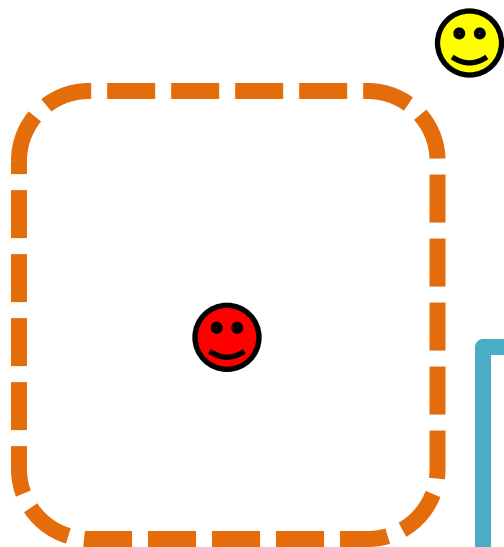
gold = 0



ทำงานใน **critical**
Section คือขโมยทอง
gold-- จะเกิด ???



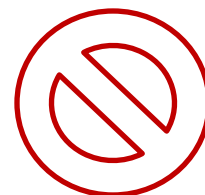


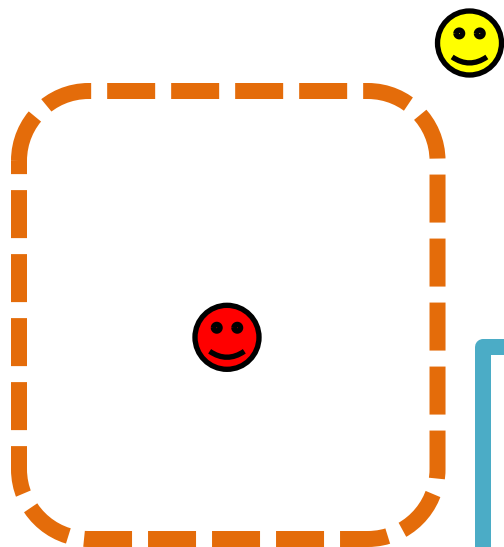


gold = 0



ทำงานใน **critical section**
ส่วนที่ 2 และจะเกิดความผิดพลาด
เนื่องจากคิดว่า เช็คแล้วว่ามีทอง





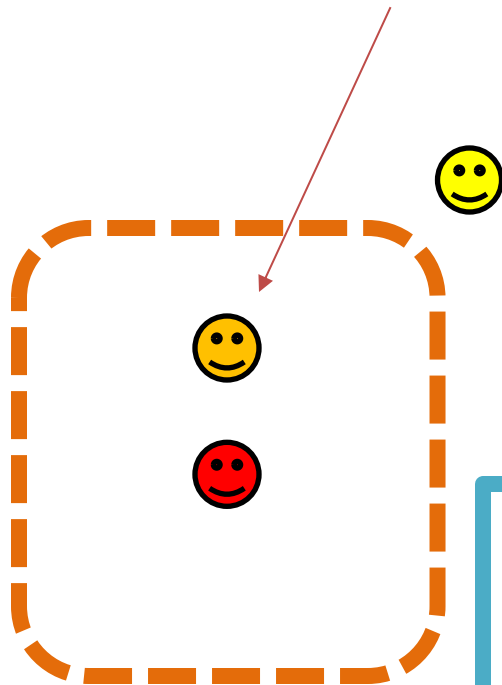
gold = 0



ดังนั้นจึงต้องเช็คอีกครั้งหนึ่งว่า
มีทองจริงหรือไม่ ก่อนทำ **critical**
section ส่วนที่ 2



เมื่อเช็คแล้วพบว่าไม่มีทองก็ต้องออกมารอ



gold = 0



```
29 void * bar(void * arg){
30     sleep(3);
31     pthread_mutex_lock(&mutex_var);
32     global_var ++;
33     printf("bar() update global_var to %d\n", global_var);
34     pthread_cond_broadcast(&cv);
35     pthread_mutex_unlock(&mutex_var);
36
37     pthread_exit((void *)NULL);
38 }
```

Operation on foo() (ตอน 2)

- ใน slide ก่อนหน้า เราเปลี่ยน `pthread_cond_signal(&cv)` ใน `bar()` เป็น `pthread_cond_broadcast(&cv)` ความแตกต่างคือ
- `pthread_cond_signal(&cv)` ปลุกเธรดที่รออยู่โดย `pthread_cond_wait()` เพียงเธรดเดียว
- แต่ `pthread_cond_broadcast(&cv)` ปลุกเธรดทุกเธรดที่รออยู่ `pthread_cond_wait()` ทุกเธรด
- ถึงจะทุกเธรดจะถูกปลุกโดย broadcast เธรดเหล่านั้นยังต้องแข่งกันเพื่อแย่ง `lock(mutex_var)` และมีเพียงเธรดเดียวเท่านั้นที่จะ return จาก `pthread_cond_wait()` ได้
- ความต่างคือนโยบายในการ scheduling ของเธรด broadcast เปิดโอกาสให้มีการแข่งขันมากกว่า signal

```
40 #define NUM_THREADS 2
41
42 int main(void){
43
44     pthread_t ntid[NUM_THREADS];
45     void *tret;
46
47     pthread_mutex_init(&mutex_var, NULL);
48     pthread_cond_init(&cv, NULL);
49
50     pthread_create(&ntid[0], NULL, foo, NULL);
51     pthread_create(&ntid[1], NULL, bar, NULL);
52
53     pthread_join(ntid[0], tret);
54     pthread_join(ntid[1], tret);
55
56     pthread_cond_destroy(&cv);
57     pthread_mutex_destroy(&mutex_var);
58     printf("main() global_var is %d\n", global_var);
59 }
```

Classic Synchronization Problem

- Bounded Buffer
- Reader-Writer Lock
- Dining Philosopher

ปัญหา Bounded Buffer

- มี Queue เป็น shared global variables
- มี mutex 1 mutex และตัวแปร condition สองตัวแปร

- Producer จะใส่ข้อมูลเข้าสู่ Queue และหยุดใส่เมื่อคิวเต็ม
- Producer จะรอจนกระทั่งคิวไม่เต็ม (consumer นำข้อมูลออกไป) แล้วจึงทำงานต่อ
- Producer จะแจ้ง consumer เสมอเมื่อมันใส่ข้อมูลสู่คิวเสร็จ

- Consumer จะนำข้อมูลออกจาก Queue และหยุดนำออกเมื่อคิวว่าง
- Consumer จะรอจนกระทั่งคิวไม่ว่าง (producer ใส่ข้อมูลเข้าสู่คิว) แล้วจึงทำงานต่อ
- Consumer จะแจ้ง producer เสมอเมื่อมันนำข้อมูลออกจากคิว

Producer Consumer

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 #define Q_SIZE 5
7 int Q[Q_SIZE];
8 int Q_len;
9 int Q_head, Q_tail;
10
11 pthread_mutex_t mutexQ;
12 pthread_cond_t not_full;
13 pthread_cond_t not_empty;
```

```
15 void init_Q(void){
16     int i;
17     Q_len = 0;
18     for(i = 0; i < Q_SIZE; i++){
19         Q[i] = 0;
20     }
21     Q_head = Q_tail = 0;
22     pthread_mutex_init(&mutexQ, NULL);
23     pthread_cond_init(&not_full, NULL);
24     pthread_cond_init(&not_empty, NULL);
25 }
26
27 void print_Q(char * prefix){
28     int i;
29     printf("%s: Q[%02d] = ( ", prefix, Q_len);
30     for(i = 0; i < Q_SIZE; i++){
31         printf("%d ", Q[i]);
32     }
33     printf(") [h %02d:t %02d]\n", Q_head, Q_tail);
34 }
35
```

```
36 int en_Q(int data){ // enqueue the queue
37
38     pthread_mutex_lock(&mutexQ);
39
40     while(Q_len == Q_SIZE){ // the Q is full.
41         print_Q("F:");
42         pthread_cond_wait(&not_full, &mutexQ);
43     }
44     Q[Q_tail] = data;
45     printf("Q[t%02d] <-- %d\n", Q_tail, data);
46     fflush(stdout);
47     Q_tail++;
48     Q_tail = Q_tail % Q_SIZE;
49     Q_len++;
50
51     pthread_cond_signal(&not_empty);
52     pthread_mutex_unlock(&mutexQ);
53     return data;
54 }
55
```

Function en_Q

- เรียก lock(mutexQ) เพื่อเข้าสู่ critical section 1
- ใน critical section 1 เช็คค่า Queue Q[] เต็มหรือไม่ (Q_len == Q_SIZE) ตราบใดที่เต็มก็จะเรียก pthread_cond_wait() เพื่อ
 - หยุดรอตัวแปร CV “not_full” ที่เทรดจะถูกปลุกก็ต่อเมื่อเกิดการนำข้อมูลออกจาก Q[]
 - Unlock(mutexQ) ให้เทรดอื่นเอาไปใช้
- ในกรณีที่เงื่อนไขข้างต้นไม่เป็นจริง (เทรดอาจไม่ได้เรียก pthread_cond_wait() หรือเรียกแล้วและรีเทรดรินออกมาจาก pthread_cond_wait() ก็ได้) ก็ให้เพิ่มค่าเข้าสู่ท้ายคิวที่ Q[tail]

Function en_Q

- $Q[\text{tail}]$ คือตำแหน่งท้ายสุดของ Q
- ให้เลื่อน tail ซึ่งเป็น index ของ element ล่าสุดของ $Q[]$ ไปอีก 1 Q_tail++
- ในกรณีที่ค่าของ Q_tail เท่ากับ Q_size เราต้อง modulo ค่าให้กลับมาเริ่มที่ 0 ใหม่
- เพิ่มขนาดของ $Q[]$ ขึ้นอีก 1 คือ Q_size++
- ส่งสัญญาณเพื่อปลุกเทรดที่รอให้มีข้อมูลเข้าสู่ $Q[]$ โดยรอผ่านตัวแปร not_empty ว่ามีการนำข้อมูลเข้า $Q[]$ แล้ว
- $\text{Unock}(\text{mutex}Q)$

```
56 int de_Q(void){ // dequeue the queue
57     int data;
58     pthread_mutex_lock(&mutexQ);
59
60     while(Q_len == 0){ // the Q is full.
61         print_Q("E:");
62         pthread_cond_wait(&not_empty, &mutexQ);
63     }
64     data = Q[Q_head];
65     printf("Q[h%02d] --> %d\n", Q_head, data);
66     fflush(stdout);
67     Q[Q_head] = 0;
68     Q_head++;
69     Q_head = Q_head % Q_SIZE;
70     Q_len--;
71
72     pthread_cond_signal(&not_full);
73     pthread_mutex_unlock(&mutexQ);
74     return data;
75 }
```

Function de_Q

- เรียก lock(mutexQ) เพื่อเข้าสู่ critical section 1
- ใน critical section 1 เช็ค Queue Q[] ว่าว่างหรือไม่ (Q_len == 0) ตราบใดที่ว่างก็จะเรียก pthread_cond_wait() เพื่อ
 - หยุดรอตัวแปร CV “not_empty” ที่เทรดจะถูกปลุกก็ต่อเมื่อเกิดการนำข้อมูลเข้าสู่ Q[]
 - Unlock(mutexQ) ให้เทรดอื่นเอาไปใช้
- ในกรณีที่เงื่อนไขข้างต้นไม่เป็นจริง (เทรดอาจไม่ได้เรียก pthread_cond_wait() หรือเรียกแล้วและรีเทรดออกมาจาก pthread_cond_wait() ก็ได้) ก็ให้นำค่าออกจากต้นคิวที่ Q[head]

Function de_Q

- $Q[\text{head}]$ คือตำแหน่งเริ่มต้น Q
- นำข้อมูลจาก $Q[\text{head}]$ ไปไว้ที่ตัวแปร data
- Clear ข้อมูลที่ตำแหน่ง $Q[\text{head}]$
- ให้เลื่อน Q_head ซึ่งเป็น index ของ element เริ่มต้นของ $Q[]$ ไปอีก 1
 Q_head++
- ในกรณีที่ค่าของ Q_head เท่ากับ Q_size เราต้อง modulo ค่าให้กลับมาเริ่มที่ 0 ใหม่
- เพิ่มขนาดของ $Q[]$ ลดลง 1 คือ Q_size--
- ส่งสัญญาณเพื่อปลุกเทรดเดอร์ให้มีการนำข้อมูลออกจาก $Q[]$ โดยรอผ่านตัวแปร not_full ว่ามีการนำข้อมูลออกจาก $Q[]$ แล้ว
- $\text{Unock}(\text{mutex}Q)$

```
77 void * producer(void * arg){
78     long data = (long) arg;
79     int i;
80     for(i = 0; i < 10; i++){
81         printf("p:");
82         en_Q(data);
83     }
84     pthread_exit((void *)NULL);
85 }
86
87 void * consumer(void * arg){
88     int data;
89     int i;
90     for(i = 0; i < 10; i++){
91         if(i % Q_SIZE == 0) sleep(2); // slowdown this thr
92         printf("c:");
93         data = de_Q();
94     }
95     pthread_exit((void *)NULL);
96 }
```

```
98 #define NUM_THREADS 2
99
100 int main(void){
101
102     pthread_t ntid[NUM_THREADS];
103     void *tret;
104
105     init_Q();
106
107     pthread_create(&ntid[0], NULL, producer, (void *)1);
108     pthread_create(&ntid[1], NULL, consumer, NULL);
109
110     pthread_join(ntid[0], tret);
111     pthread_join(ntid[1], tret);
112
113     pthread_cond_destroy(&not_full);
114     pthread_cond_destroy(&not_empty);
115     pthread_mutex_destroy(&mutexQ);
116 }
```

```
openstack@source:~/system-programming/pthread$ gcc -o e13 mutexex13-producer-consumer-pfast.c -lpthread
openstack@source:~/system-programming/pthread$ ./e13
p:Q[t00] <-- 1
p:Q[t01] <-- 1
p:Q[t02] <-- 1
p:Q[t03] <-- 1
p:Q[t04] <-- 1
p:F:: Q[05] = ( 1 1 1 1 1 ) [h 00:t 00]
c:Q[h00] --> 1
c:Q[h01] --> 1
c:Q[h02] --> 1
c:Q[h03] --> 1
c:Q[h04] --> 1
Q[t00] <-- 1
p:Q[t01] <-- 1
p:Q[t02] <-- 1
p:Q[t03] <-- 1
p:Q[t04] <-- 1
c:Q[h00] --> 1
c:Q[h01] --> 1
c:Q[h02] --> 1
c:Q[h03] --> 1
c:Q[h04] --> 1
```

pthread_cond_broadcast()

- สามารถใช้แทนที่ pthread_cond_signal() ได้

```
int pthread_cond_broadcast(pthread_cond_t *cond);  
int pthread_cond_signal(pthread_cond_t *cond);
```

- จะปลุก thread ทุก thread ที่รอตัวแปร conditional variable “cond” อยู่ ให้ตื่นขึ้นมา แข่งเอา mutual exclusion variable

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <pthread.h>
5
6  #define Q_SIZE 5
7  int Q[Q_SIZE];
8  int Q_len;
9  int Q_head, Q_tail;
10
11 pthread_mutex_t mutexQ;
12 pthread_cond_t not_full;
13 pthread_cond_t not_empty;
14
```

```
15  int items_per_prod;
16  int items_per_cons;
17
18  void init_Q(void){
19      int i;
20      Q_len = 0;
21      for(i = 0; i < Q_SIZE; i++){
22          Q[i] = 0;
23      }
24      Q_head = Q_tail = 0;
25      pthread_mutex_init(&mutexQ, NULL);
26      pthread_cond_init(&not_full, NULL);
27      pthread_cond_init(&not_empty, NULL);
28  }
29
30  void print_Q(int prefix){
31      int i;
32      printf("%d: Q[%02d] = ( ", prefix, Q_len);
33      for(i = 0; i < Q_SIZE; i++){
34          printf("%d ", Q[i]);
35      }
36      printf(") [h %02d:t %02d]\n",Q_head, Q_tail);;
37  }
```

```
39  int en_Q(int data){ // enqueue the queue
40
41      pthread_mutex_lock(&mutexQ);
42
43      while(Q_len == Q_SIZE){ // the Q is full.
44          print_Q(1);
45          pthread_cond_wait(&not_full, &mutexQ);
46      }
47      Q[Q_tail] = data;
48      printf("Q[t%02d]Ql=%d <-- %d\n", Q_tail, Q_len, data);
49      Q_tail = (Q_tail + 1) % Q_SIZE;
50      Q_len++;
51      pthread_cond_broadcast(&not_empty);
52      pthread_mutex_unlock(&mutexQ);
53      return data;
54  }
55  --
```

```
56  int de_Q(void){ // dequeue the queue
57      int data;
58      pthread_mutex_lock(&mutexQ);
59
60      while(Q_len == 0){ // the Q is full.
61          print_Q(0);
62          pthread_cond_wait(&not_empty, &mutexQ);
63      }
64      data = Q[Q_head];
65      printf("Q[h%02d]Ql=%d --> %d\n", Q_head, Q_len, data);
66      Q[Q_head] = 0;
67      Q_head = (Q_head + 1) % Q_SIZE;
68      Q_len--;
69
70      pthread_cond_broadcast(&not_full);
71      pthread_mutex_unlock(&mutexQ);
72      return data;
73 }
```

```
75 void * producer(void * arg){
76     long id = (long) arg;
77     int data = 1;
78     int i;
79     for(i = 0; i < items_per_prod; i++){
80         printf("p%ld(%d):", id, i);
81         en_Q(data);
82     }
83 }
84
85 void * consumer(void * arg){
86     long id = (long) arg;
87     int data;
88     int i;
89     for(i = 0; i < items_per_cons; i++){
90         printf("c%ld(%d):", id, i);
91         data = de_Q();
92     }
93 }
```

```
95  #define CONSUME_UNIT 10
96
97  int main(int argc, char * argv[]){
98      int i;
99      int num_prod = 1;
100     int num_cons = 3;
101     pthread_t *ptid;
102     pthread_t *ctid;
103
104     printf("argc = %d\n", argc);
105     if(argc == 1){
106         num_prod = 1;
107         num_cons = 3;
108         items_per_prod = num_cons * CONSUME_UNIT;
109         items_per_cons = CONSUME_UNIT;
110     }
111     else if(argc == 2){
112         num_prod = 1;
113         num_cons = atoi(argv[1]);
114         items_per_prod = num_cons * CONSUME_UNIT;
115         items_per_cons = CONSUME_UNIT;
116     }
117     else if(argc == 3){
118         num_prod = atoi(argv[1]);
119         num_cons = atoi(argv[2]);
120         items_per_cons = CONSUME_UNIT;
121         items_per_prod = num_cons * CONSUME_UNIT;
122         if(items_per_prod % num_prod == 0){
123             items_per_prod = (int)(items_per_prod/num_prod);
124             printf("items per producer = %d, items per consumers = %d\n",
125                 items_per_prod, items_per_cons);
126         }
```

```
127     else{
128         printf("Since each consumer consume %d items,\nthe total number of items needed by
129             items_per_cons, items_per_prod, num_prod);
130         exit(1);
131     }
132 }
133 else{
134     printf("wrong input parameters\n");
135     exit(1);
136 }
---
```

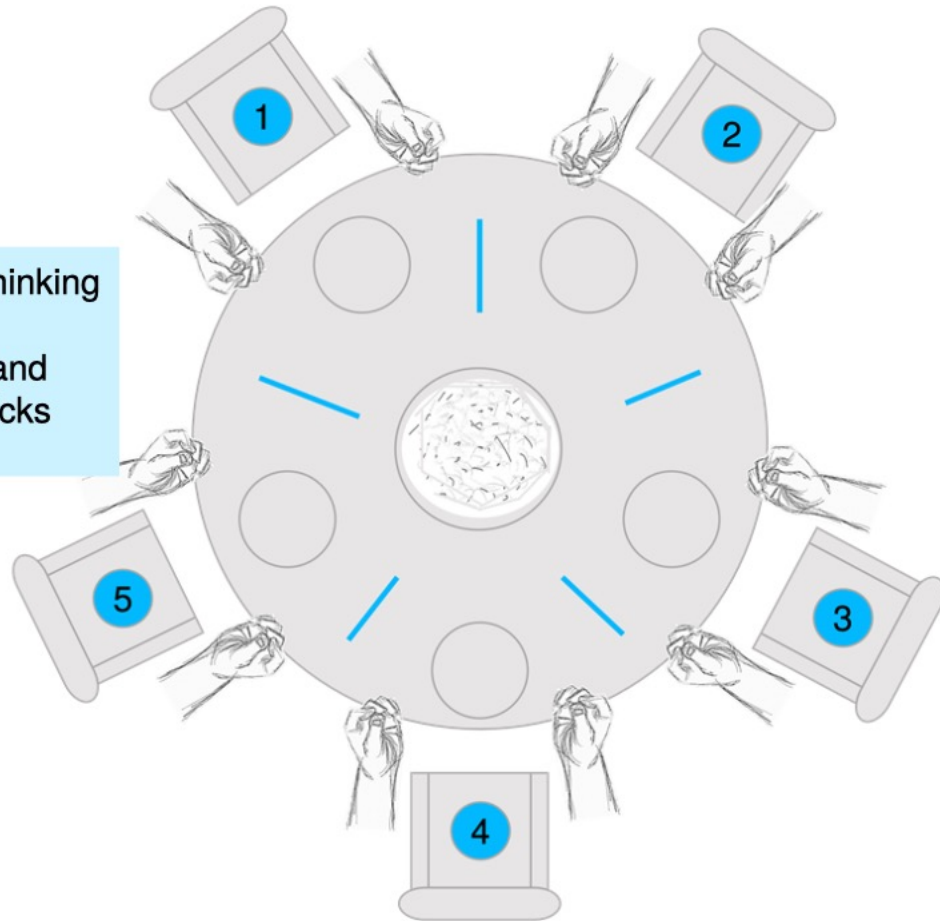
```
138     ptid = (pthread_t *)malloc(num_prod*sizeof(pthread_t));
139     ctid = (pthread_t *)malloc(num_cons*sizeof(pthread_t));
140
141     void *tret;
142
143     init_Q();
144
145     for(i=0; i < num_prod; i++){
146         pthread_create(ptid+i, NULL, producer, (void *) (long)i);
147     }
148     for(i=0; i < num_cons; i++){
149         pthread_create(ctid+i, NULL, consumer, (void *) (long)i);
150     }
151
152     for(i=0; i < num_prod; i++){
153         pthread_join(ptid[i], NULL);
154     }
155     for(i=0; i < num_cons; i++){
156         pthread_join(ctid[i], NULL);
157     }
158
159     pthread_cond_destroy(&not_full);
160     pthread_cond_destroy(&not_empty);
161     pthread_mutex_destroy(&mutexQ);
162
163 }
```

แบบฝึกหัด 1

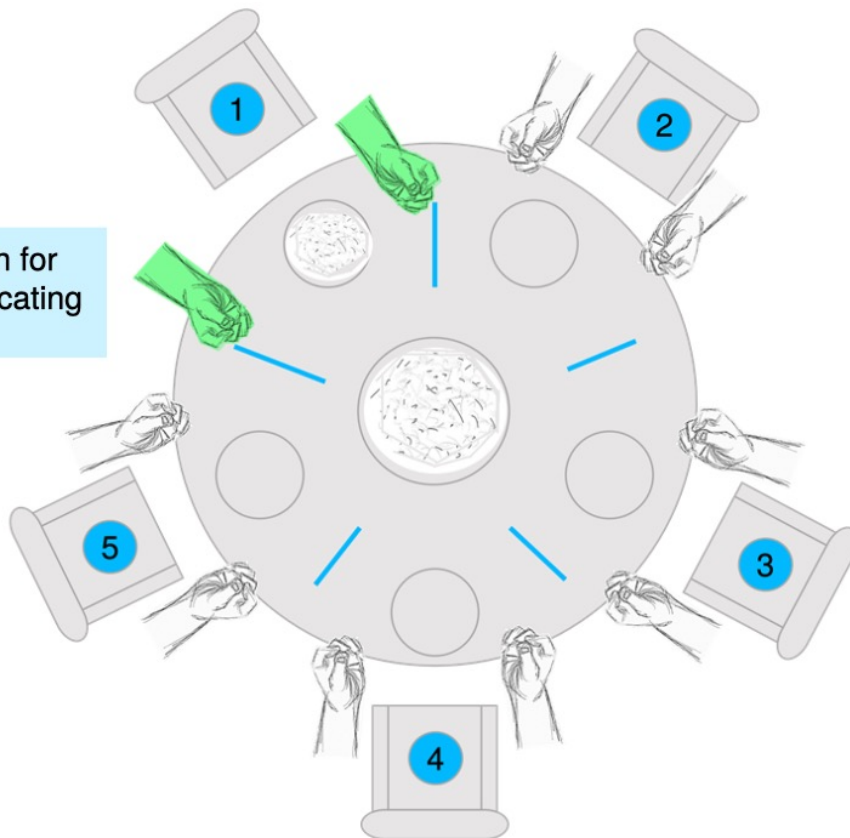
- ขอให้ นศ เขียนรายงานวิเคราะห์โครงสร้างของโปรแกรม
- mutexex16-producer-consumer-MpMc.c และ
- mutexex17-producer-consumer-broadcast-MpMc.c
- ขอให้อธิบายการใช้งาน condition variables ของโปรแกรมทั้งสอง
- ขอให้อธิบายว่า signal และ broadcast ต่างกันอย่างไร
- ขอให้ compile และรันโปรแกรมทั้งสอง โดยกำหนดให้ใช้ 5 producer threads และ 5 consumer threads
- ขอให้ capture ผลและนำมาใส่ในรายงาน

ปัญหา Dining Philosopher

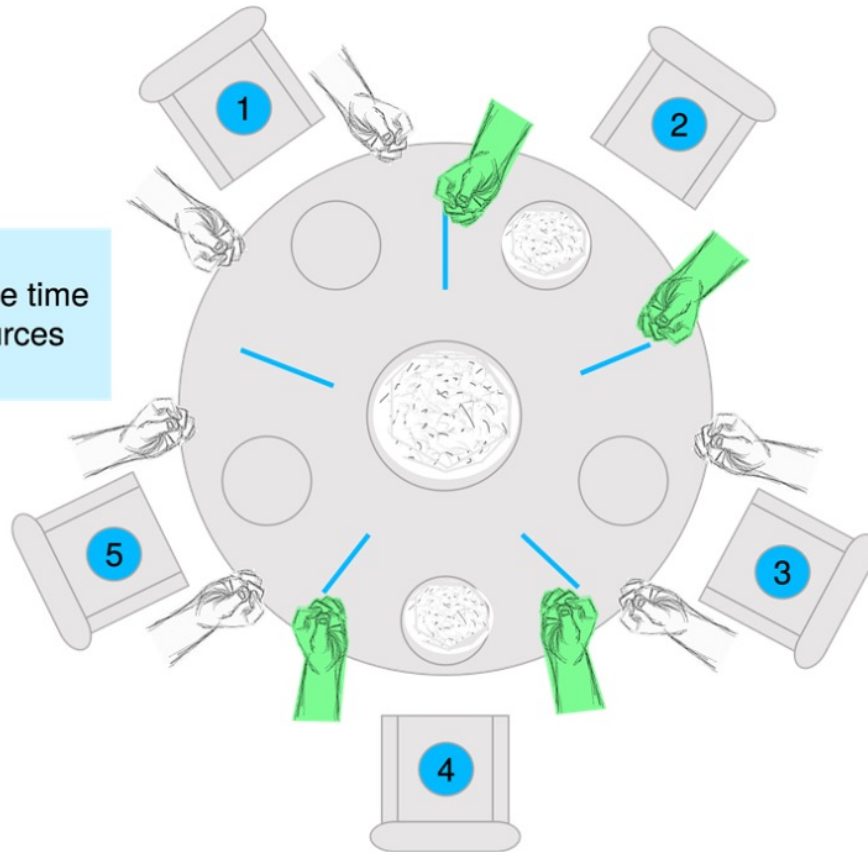
five philosophers sit at a table, thinking
each has a chopstick to the left and
one to the right, with five chopsticks
total



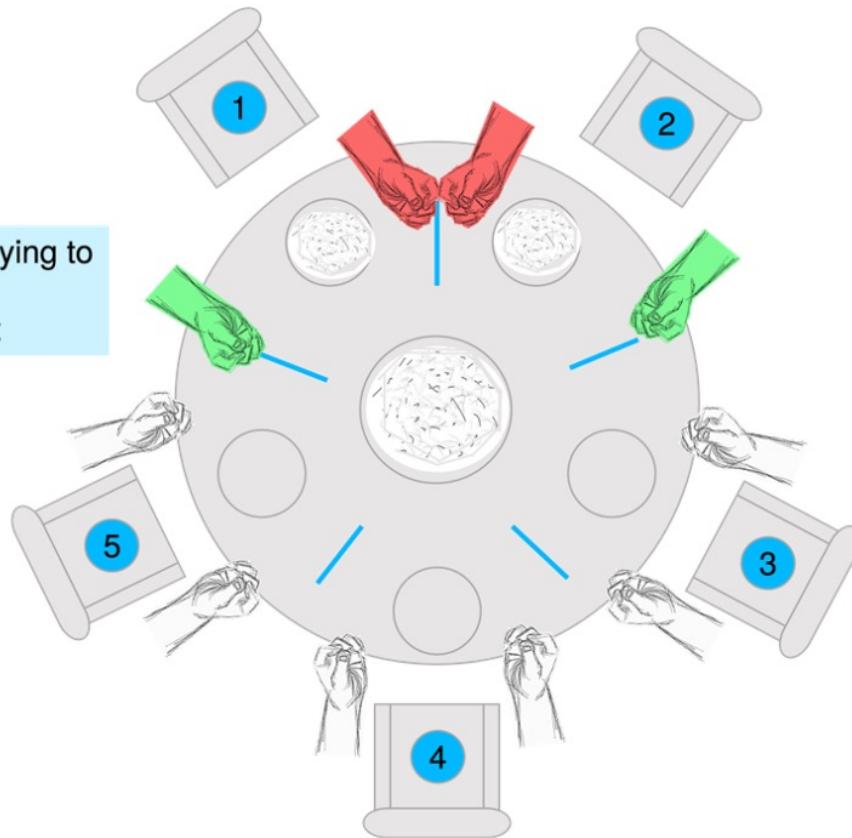
if one (1) gets hungry, they reach for the left and right chopsticks, allocating both



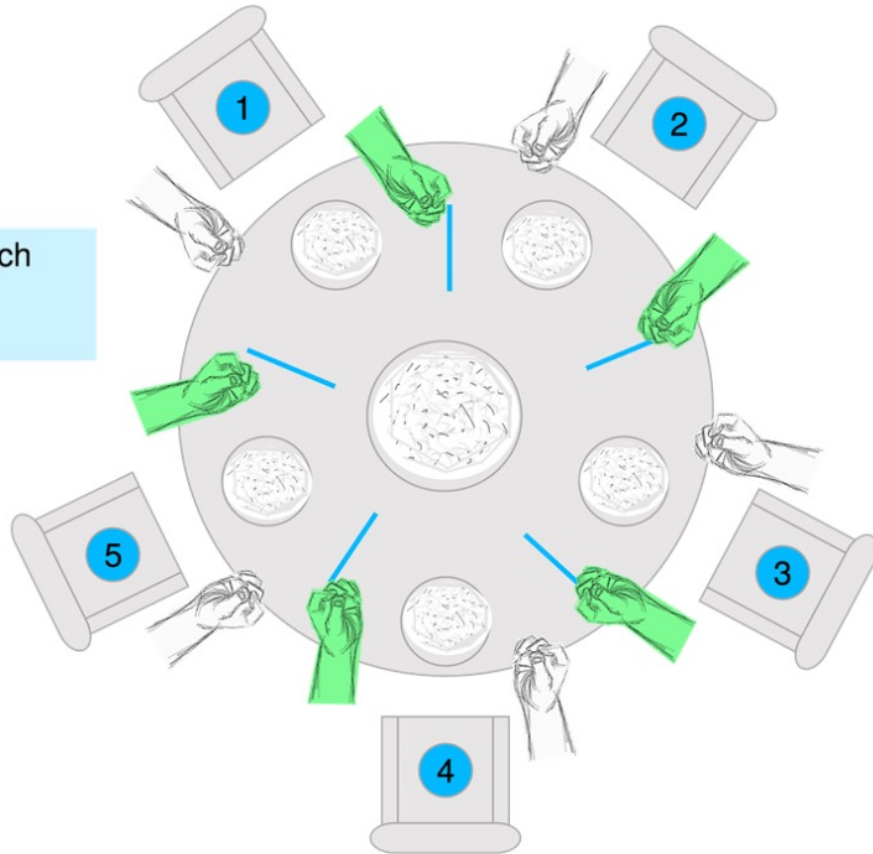
two or more non-adjacent
philosophers can eat at the same time
as they do not contend for resources
(e.g. 2 and 4)



but two adjacent philosophers trying to eat (e.g. 1 and 2) contend for resources, and only one can eat



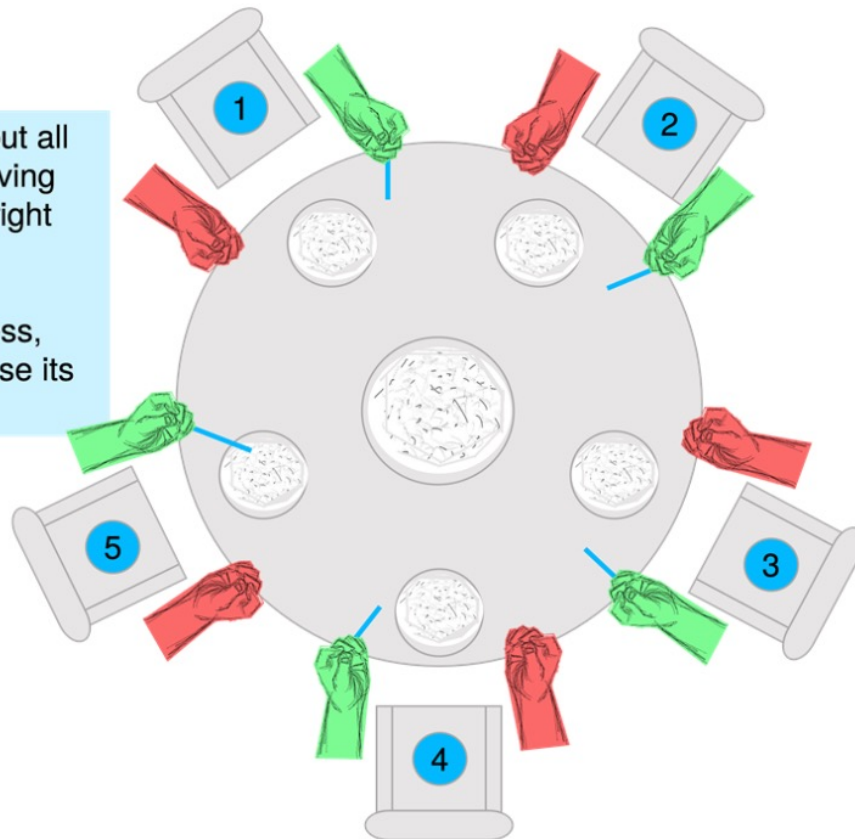
if they all get hungry at once, each reaches with their left hand (for example) for a chopstick



each grasps the left chopstick, but all the chopsticks are allocated, leaving none free for any philosopher's right hand

no philosopher can make progress, therefore none will eat and release its resources

deadlock



pthread_barrier_wait()

```
#include <pthread.h>
int pthread_barrier_init (pthread_barrier_t *barrier,
                        const pthread_barrierattr_t *attr,
                        unsigned int count);
```

- ตัวแปร pthread_barrier_t *barrier
- ทุกเธรดจะเรียกใช้ interface นี้
- ตราบใดที่จำนวนเธรดที่เรียกใช้ interface นี้ไม่เกิน count เธรดก็จะ block รอ
- เมื่อจำนวนเธรดที่เรียกใช้ interface นี้เท่ากับ count เธรดทุกเธรดที่เรียกใช้ interface ก็จะ unblock และประมวลผลคำสั่งถัดไป

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  #define THREAD_NUMS 4
5  pthread_barrier_t barrier;
6
7  void *t0(void *param)
8  {
9      pthread_barrier_wait(&barrier);
10     printf("t0 ready\n");
11 }
12 void *t1(void *param)
13 {
14     pthread_barrier_wait(&barrier);
15     printf("t1 ready\n");
16 }
17 void *t2(void *param)
18 {
19     pthread_barrier_wait(&barrier);
20     printf("t2 ready\n");
21 }
```

```
23  int main(void)
24  {
25      pthread_t t[3];
26
27      pthread_barrier_init(&barrier, NULL, THREAD_NUMS);
28
29      pthread_create(&t[0], NULL, t0, NULL);
30      pthread_create(&t[1], NULL, t1, NULL);
31      pthread_create(&t[2], NULL, t2, NULL);
32
33      pthread_barrier_wait(&barrier);
34      printf("all sub threads ready, go!\n");
35
36      pthread_barrier_destroy(&barrier);
37
38  }
```

อ้างอิง <https://gist.github.com/shelterz/4b13459668eec743f15be6c200aa91b2>

แบบฝึกหัด 2

- ขอให้ นศ ออกแบบและเขียนโปรแกรมที่รับค่า N เป็น parameter
- ให้มี shared global variable ได้แก่
 - int tcount; เริ่มต้น = 0
 - ตัวแปร condition variable ชื่อ CV
 - ตัวแปร mutex variable ชื่อ barr
- ใน main() กำหนดให้ tcount = 0 และ nflag = 0 ก่อนสร้างเธรด
- โปรแกรมมีเธรด N = 10 เธรด แต่ละเธรด เรียกใช้ function thr() โดย
ต้องที่ในการสร้างเธรดด้วย pthread_create()
- Function thr() เรียก function barrier() (ทุกเธรดทำเหมือนกัน)

function barrier()

- เรียก lock(bar) ถ้าเทรตผ่าน lock(bar) ก็เข้าสู่ critical section 1
- Critical section 1 จะเพิ่มค่า tcount++;
- หลังจากนั้นเทรตจะเช็คค่า tcount < N หรือไม่ ตราบใดที่เงื่อนไขนี้เป็นจริง เทรตจะหยุดรอ ด้วยคำสั่ง pthread_cond_wait() ซึ่งจะทำสิ่งต่อไปนี้
 - หยุดเพื่อรอการปลุกผ่าน cond variable “CV”
 - Unlock mutex “bar” ให้เทรตอื่นได้เข้า critical section ของตน

function barrier()

- ในกรณีที่เงื่อนไขข้างต้นไม่จริง (ค่า `tcount == N`) เธรดจะเข้าสู่ critical section 2 เพื่อทำสิ่งต่อไปนี้
 - เรียก `pthread_cond_broadcast(&CV)` เพื่อปลุกทุกเธรด
 - พิมพ์ค่า `tcount` ออกหน้าจอ
 - ใช้คำสั่ง `unlock` เพื่อ unlock mutex barr
- ขอให้หาข้อมูลจาก internet และอธิบายในรายงานว่า interface “`pthread_barrier()`” คืออะไรใช้ทำอะไร
- จงอธิบายว่าในภาพรวมแล้ว `function barrier` ทำอะไร

Implementation ของ Mutex lock/unlock ในระบบ Linux

- ใช้ futex และ spinlock
- ดูคำอธิบาย code ได้ที่

<https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf>

mutex_lock

```
1 void mutex_lock (int *mutex) {
2     int v;
3     // Bit 31 was clear, we got the mutex (fastpath)
4     if (atomic_bit_test_set (mutex, 31) == 0)
5         return;
6     atomic_increment (mutex);
7     while (1) {
8         if (atomic_bit_test_set (mutex, 31) == 0) {
9             atomic_decrement (mutex);
10            return;
11        }
12        // Have to waitFirst to make sure futex value
13        // we are monitoring is negative (locked).
14        v = *mutex;
15        if (v >= 0)
16            continue;
17        futex_wait (mutex, v);
18    }
19 }
```

mutex_unlock

```
21 void mutex_unlock (int *mutex) {
22     // Adding 0x80000000 to counter results in 0 if and
23     // only if there are not other interested threads
24     if (atomic_add_zero (mutex, 0x80000000))
25         return;
26
27     // There are other threads waiting for this mutex,
28     // wake one of them up.
29     futex_wake (mutex);
30 }
```

Summary

- เราได้ศึกษา Thread synchronization
 - แบบ Conditional variables
- ศึกษาโปรแกรม producer consumer
- ศึกษา barrier synchronization
- เราสามารถใช้ conditional variable สร้าง barrier synchronization ได้
- ไม่ได้ศึกษา *Reader Writer Synchronization*